pygeoinf

Technical Manual

Theory, Algorithms, and API Reference

Adrian

Department of Earth Sciences

March 2, 2026

Preface

Who this manual is for

This manual is written for geophysicists who have a forward operator and a dataset and want to perform rigorous linear inversion or inference. The word *rigorous* carries weight here: pygeoinf does not cut corners. It insists on correct Hilbert space structure, proper adjoints, and treatment of uncertainty that is mathematically honest. If you are used to inverting $(\mathbf{G}^\top\mathbf{G})^{-1}\mathbf{G}^\top$ and calling it done, this manual will explain why that is sometimes wrong, and what to do instead.

No prior knowledge of functional analysis is assumed, but you will need to engage with the mathematics. The package deliberately surfaces conceptual machinery — Riesz maps, dual pairings, Hilbert adjoints — that many inversion codes hide. The reward is a library whose results are correct by construction for *any* choice of model space, inner product, and forward operator, not just the Euclidean special case.

How to read this manual

Each chapter follows a consistent four-part structure:

1. *Why this matters* — a motivating argument, usually with a geophysical example, that explains why the concept is worth the effort. Highlighted in yellow callout boxes.
2. *Definitions and theorems* — the full, rigorous mathematical treatment.
3. *Implementation in pygeoinf* — how each mathematical concept maps to the API. Highlighted in blue callout boxes labelled "pygeoinf API".
4. *Worked example* — a complete mini-example, in both mathematics and code. Highlighted in green callout boxes.

If you are primarily a practitioner, you can read the yellow and blue boxes and skip the proofs on a first pass without losing the ability to use the package. If you are mathematically inclined, read everything; proofs that would interrupt the flow are deferred to Appendix C.

Red callout boxes labelled "Common Pitfall" flag mistakes that are easy to make and important to avoid.

Acknowledgements

This manual documents work carried out at the Department of Earth Sciences. The theoretical framework draws heavily on Al-Attar and Crawford [1], Stuart [5], and the trilogy Backus [2, 3, 4].

Contents

Preface **iii**

Notation and Conventions **xi**

I Foundations **1**

1 Introduction and Motivation **3**
1.1 What is pygeoinf? . . . 3
1.2 The geophysical inversion problem . . . 3
1.3 Why abstract spaces matter . . . 4
1.4 Package architecture overview . . . 5
1.5 How to read this manual . . . 6
1.6 Notation and conventions summary . . . 6

2 Hilbert Spaces **9**
2.1 Why Hilbert spaces? . . . 9
2.2 Definition and axioms . . . 9
2.3 Inner products, norms, and distances . . . 10
2.4 The dual space and dual pairing . . . 10
2.5 The Riesz representation theorem . . . 11
2.6 Riesz-first design in pygeoinf . . . 12
2.7 Bases and Representations: The Primal Side . . . 13
2.7.1 Synthesis: From Coefficients to Vectors . . . 13
2.7.2 Analysis: From Vectors to Coefficients . . . 14
2.8 The Gram Matrix: The Metric Hidden in the Basis . . . 14
2.8.1 The Induced Coefficient Inner Product . . . 15
2.8.2 The Gram Operator . . . 15
2.8.3 Why the Gram Matrix Matters . . . 15
2.9 Duality and the Riesz Map in Coordinates . . . 16
2.9.1 What Happens in Coordinates . . . 16
2.10 Finite-Dimensional Approximation . . . 18
2.10.1 Finite Synthesis and Analysis . . . 18
2.10.2 The Finite Gram Matrix . . . 19
2.10.3 Properties of the Finite Gram Matrix . . . 19
2.10.4 Finite Analysis via the Gram Matrix . . . 19
2.10.5 Finite Riesz Map in Coordinates . . . 20

2.10.6 Summary: What Changes in Finite Dimensions . . . 20
2.11 Concrete spaces in pygeoinf . . . 20
2.11.1 `EuclideanSpace` . . . 21
2.11.2 `MassWeightedHilbertSpace` . . . 21
2.11.3 `HilbertSpaceDirectSum` . . . 21
2.11.4 Implementing your own `HilbertSpace` . . . 21

3 Linear Forms **23**
3.1 Motivation: measurements as linear functionals . . . 23
3.2 Definition . . . 23
3.3 Representation in coordinates . . . 24
3.4 The Riesz representative (gradient) . . . 24
3.5 Interoperability: viewing a linear form as an operator . . . 25

4 Linear Operators **27**
4.1 Motivation: the forward operator . . . 27
4.2 Definition and properties . . . 27
4.3 The Banach dual (transpose) G' . . . 28
4.4 The Hilbert adjoint G^* . . . 28
4.5 Matrix representations . . . 29
4.5.1 The standard (component) matrix . . . 30
4.5.2 The Galerkin matrix . . . 30
4.5.3 The discrete adjoint formula . . . 31
4.6 Worked example: `from_matrix`, dual vs adjoint, and Galerkin matrices . . . 31
4.7 Operators as stacks of linear forms . . . 33
4.8 Constructing operators in pygeoinf . . . 33
4.9 Advanced routines: randomized spectral surrogates . . . 35
4.10 The full commutative diagram . . . 36

5 Nonlinear Operators and Forms **39**
5.1 NonLinearOperator . . . 39
5.2 NonLinearForm . . . 39
5.3 Gradients and Hessians . . . 39

II Geometry and Structure **41**

6 Subsets and Convex Sets **43**
6.1 Why convex sets? . . . 43
6.2 The Subset interface . . . 43
6.3 Balls and ellipsoids . . . 43
6.4 Half-spaces and polyhedral sets . . . 43
6.5 Degenerate sets: EmptySet and FullSpace . . . 43
6.6 Set operations . . . 43

7 Subspaces **45**
7.1 Motivation: equality constraints . . . 45
7.2 Orthogonal projectors . . . 45
7.3 Linear subspaces . . . 45

7.4 Affine subspaces . . . 45
7.4.1 Construction from tangent bases . . . 45
7.4.2 Construction from linear equations . . . 45
7.5 Tangent spaces and dimension . . . 45

8 Convex Analysis and Support Functions 47
8.1 Why convex analysis? . . . 47
8.2 Support functions: intuition . . . 48
8.3 Formal definition and properties . . . 48
8.4 Subgradients and support points . . . 48
8.5 Concrete support functions . . . 48
8.5.1 BallSupportFunction . . . 48
8.5.2 EllipsoidSupportFunction . . . 48
8.5.3 PolyhedralSupportFunction . . . 48
8.5.4 SobolevBallSupportFunction . . . 48
8.6 Combinators: building complex sets from simple ones . . . 48
8.6.1 Infimal convolution (Minkowski sum) . . . 48
8.6.2 Support function sum (intersection) . . . 48
8.6.3 Scaled sums . . . 48
8.7 The dual master equation . . . 48

9 Gaussian Measures 49
9.1 Why Gaussian measures? . . . 49
9.2 Gaussian measures on Hilbert spaces . . . 49
9.3 Sampling . . . 49
9.4 Push-forward and marginals . . . 49
9.5 Conditioning (Bayesian update) . . . 49

III Problems and Algorithms 51

10 Forward Problems 53
10.1 The forward modelling framework . . . 53
10.2 ForwardProblem and LinearForwardProblem . . . 53

11 Deterministic Inversion 55
11.1 Least-squares inversion . . . 55
11.2 Minimum-norm inversion . . . 55
11.3 Constrained inversions . . . 55
11.4 Connection to the dual master equation . . . 55

12 Bayesian Inversion 57
12.1 The Bayesian framework for inverse problems . . . 57
12.2 Linear Bayesian inversion . . . 57
12.3 Constrained Bayesian inversion . . . 57
12.4 Posterior uncertainty . . . 57

13 Backus–Gilbert and DLI Inference 59
13.1 The inference problem . . . 59

13.2 Backus–Gilbert estimators . . . 59
13.3 The admissible property set $\mathcal{U}$ . . . 59
13.4 Computing $\mathcal{U}$ via the dual master equation . . . 59
13.5 Polytope outer approximation . . . 59

14 Convex Optimisation **61**
14.1 Overview of optimisation in pygeoinf . . . 61
14.2 Linear solvers . . . 61
14.3 Convex optimisation algorithms . . . 61
14.4 Preconditioners . . . 61

IV Computational Methods and Visualisation **63**

15 Computational Utilities **65**
15.1 Randomised matrix decompositions . . . 65
15.2 Direct sum operators . . . 65
15.3 Parallel computation . . . 65
15.4 Auxiliary functions . . . 65

16 Visualisation **67**
16.1 The SubspaceSlicePlotter . . . 67
16.2 1D slices . . . 67
16.3 2D slices . . . 67
16.4 3D slices . . . 67
16.5 Worked example: visualising the admissible property set . . . 67

Appendices **71**

A Convex Analysis Primer **71**
A.1 Convex sets and functions . . . 71
A.2 Separation theorems . . . 71
A.3 Conjugate (Fenchel–Legendre) duality . . . 71
A.4 Subdifferentials . . . 71
A.5 Infimal convolution . . . 71

B Functional Analysis Prerequisites **73**
B.1 Banach spaces . . . 73
B.2 Weak and weak-* topologies . . . 73
B.3 Compact operators . . . 73
B.4 Spectral theory for self-adjoint operators . . . 73

C Deferred Proofs **75**

D Complete API Quick Reference **77**
D.1 hilbert_space module . . . 77
D.2 linear_forms module . . . 77
D.3 linear_operators module . . . 77

D.4 subsets module 77
D.5 subspaces module 77
D.6 convex_analysis module 77
D.7 gaussian_measure module 77
D.8 forward_problem module 77
D.9 inversion module 77
D.10 linear_bayesian module 77
D.11 linear_optimisation module 77
D.12 backus_gilbert module 77
D.13 visualisation module 77

Notation and Conventions

General conventions

Throughout this manual we use the following typographic conventions.

- **Boldface roman** ($\mathbf{m}$, $\mathbf{G}$, $\boldsymbol{\lambda}$) denotes finite-dimensional vectors and matrices, i.e. elements of $\mathbb{R}^n$ or linear maps $\mathbb{R}^n \to \mathbb{R}^m$. We also use boldface for coefficient vectors/sequences and concrete operators acting on coefficient spaces (e.g. $\mathbf{m} \in \ell^2$ and $\mathbf{M}_\phi$).
- **Calligraphic** ($\mathcal{M}$, $\mathcal{D}$, $\mathcal{P}$) denotes abstract (possibly infinite-dimensional) Hilbert or Banach spaces.
- **Script/Roman capitals** (G, T, Π, π, $\mathcal{R}$) denote abstract linear operators between such spaces.
- Indices: j, k, ℓ typically index a basis of the model space; i typically indexes a datum.
- We work exclusively over $\mathbb{R}$; all spaces are real.

Spaces

Symbol	Meaning	Code
$\mathcal{M}$	Model space (Hilbert)	`model_space`
$\mathcal{D}$	Data space (usually $\mathbb{R}^{N_d}$)	`data_space`
$\mathcal{P}$	Property space (usually $\mathbb{R}^{N_p}$)	`property_space`
$\mathcal{M}'$	Continuous dual of $\mathcal{M}$	implicit via Riesz
$\mathcal{D}'$	Continuous dual of $\mathcal{D}$	implicit via Riesz
H	Generic Hilbert space	`HilbertSpace`
ℓ^2	Square-summable sequences	infinite-dim. limit
$\mathbb{R}^N$	Finite coefficient space	`EuclideanSpace(N)`
$\mathcal{M}_N$	N-dimensional subspace of $\mathcal{M}$	`dim = N`
$H_1 \oplus H_2$	Hilbert direct sum	`HilbertSpaceDirectSum`

Coefficient spaces and standard bases

We use two closely related coefficient spaces:

- the infinite-dimensional space ℓ^2 of square-summable sequences, and

- its finite-dimensional truncation $\mathbb{R}^N$.

For ℓ^2 we denote the standard basis vectors by $\{e_j\}_{j=1}^{\infty}$, where $(e_j)_k = \delta_{jk}$. For $\mathbb{R}^N$ we denote the standard basis vectors by $\{\mathbf{e}_j\}_{j=1}^{N}$.

On the dual coefficient spaces we use the corresponding *coordinate functionals* $\{e^j\}_{j=1}^{\infty} \subset (\ell^2)'$ and (optionally) $\{\mathbf{e}^j\}_{j=1}^{N} \subset (\mathbb{R}^N)'$, defined by

$$e^j(\mathbf{c}) = c_j, \qquad \mathbf{e}^j(\mathbf{a}) = a_j.$$

They satisfy the canonical pairing relations

$$\langle e^i, e_j \rangle = \delta^i_j, \qquad \langle \mathbf{e}^i, \mathbf{e}_j \rangle = \delta^i_j.$$

Remark. For the standard ℓ^2 inner product $(\mathbf{c}, \mathbf{d})_{\ell^2} := \sum_j c_j d_j$, the continuous dual satisfies $(\ell^2)' \cong \ell^2$ via the Riesz map; in particular, the coordinate functional e^j corresponds to the vector e_j. We will nevertheless keep primes (e.g. $(\ell^2)'$) in the manual to make dual/primal roles explicit.

Operators and maps

Symbol	Meaning	Code
$G : \mathcal{M} \to \mathcal{D}$	Forward operator	`forward_operator`
$G' : \mathcal{D}' \to \mathcal{M}'$	Banach dual (transpose)	`.dual`
$G^* : \mathcal{D} \to \mathcal{M}$	Hilbert adjoint	`.adjoint`
$\mathcal{T} : \mathcal{M} \to \mathcal{P}$	Property operator	`property_operator`
$\mathcal{R}_X : X' \to X$	Riesz isomorphism	`from_dual`
$\mathcal{R}_X^{-1} : X \to X'$	Inverse Riesz	`to_dual`
$\Pi : \mathcal{M} \to \ell^2$	Analysis (expansion coefficients)	`to_components`
$\pi : \ell^2 \to \mathcal{M}$	Synthesis (reconstruct from coeffs)	`from_components`
$\Pi_N : \mathcal{M} \to \mathbb{R}^N$	Truncated analysis	`to_components` (finite)
$\pi_N : \mathbb{R}^N \to \mathcal{M}_N$	Truncated synthesis	`from_components` (finite)
$P_N : \mathcal{M} \to \mathcal{M}_N$	Orthogonal projection onto $\mathcal{M}_N$	implicit
$\phi_j \in \mathcal{M}$	Basis vector	`basis_vector(j)`
$\psi_j \in \mathcal{M}$	Biorthogonal dual basis vector	derived from Gram

Inner products and pairings

Symbol	Meaning	Code
$(m_1, m_2)_{\mathcal{M}}$	Inner product on $\mathcal{M}$	`inner_product(x1,x2)`
$(d_1, d_2)_{\mathcal{D}}$	Inner product on $\mathcal{D}$	`inner_product(d1,d2)`
$\langle \ell, m \rangle$	Dual pairing: $\ell \in \mathcal{M}'$, $m \in \mathcal{M}$	`l(m)`
$\lVert m \rVert_{\mathcal{M}}$	Hilbert norm $\sqrt{(m,m)_{\mathcal{M}}}$	`norm(m)`

We use *parentheses* $(\cdot, \cdot)$ for inner products and *angle brackets* $\langle \cdot, \cdot \rangle$ for dual pairings. On a Hilbert space the two are related by $\langle \ell, m \rangle = (m, \mathcal{R}_X \ell)_X$.

Matrices (finite-dimensional objects)

Symbol	Meaning	Notes
$\mathbf{M}_{\phi,N}$	Gram/mass matrix $[\,(\phi_k,\phi_j)_{\mathcal{M}}\,]_{j,k}$	SPD, $N \times N$
$\mathbf{G}$	Standard discrete matrix of G: $[\,(G\phi_j,\beta_i)_{\mathcal{D}}\,]$	$N_d \times N$
$\tilde{\mathbf{G}}$	Galerkin matrix (with inner products)	`.matrix(galerkin=True)`
$\mathbf{m},\mathbf{d}$	Coefficient vectors in $\mathbb{R}^N$, $\mathbb{R}^{N_d}$	`to_components(m)`

Measures and probability

Symbol	Meaning	Code
μ	A probability measure on $\mathcal{M}$	
$\mathcal{N}(\mu_0, C)$	Gaussian measure, mean μ_0, covariance C	`GaussianMeasure`
μ^0	Prior measure	`prior`
$\mu^{\tilde{d}}$	Posterior measure given data $\tilde{d}$	`.invert(data)`
$C : \mathcal{M} \to \mathcal{M}$	Covariance operator	`.covariance_operator`
$\mathbf{C}_{\mathcal{D}} : \mathcal{D} \to \mathcal{D}$	Data noise covariance	`data_error_measure`
$\varepsilon \in \mathcal{D}$	Data noise, $\varepsilon \sim \mathcal{N}(0, \mathbf{C}_{\mathcal{D}})$	

Convex analysis

Symbol	Meaning	Code
$C \subset \mathcal{M}$	A convex subset	`Subset` subclass
$h_C(q) = \sigma_C(q)$	Support function of C	`SupportFunction`
$\partial h_C(q)$	Subdifferential of h_C at q	`.subgradient(q)`
$\mathcal{U} \subset \mathcal{P}$	Admissible property set	computed by DLI

Part I

Foundations

Chapter 1

Introduction and Motivation

1.1 What is pygeoinf?

pygeoinf is a Python library for rigorous linear geophysical inversion and Bayesian inference. It is built around a single design philosophy: *the mathematical structure of the problem should be encoded explicitly in the code*, not reduced to bare matrices and vectors.

Most inversion codes operate on the discrete level from the start: the user assembles a matrix $\mathbf{G} \in \mathbb{R}^{M \times N}$, a data vector $\mathbf{d} \in \mathbb{R}^M$, and calls some solver. This approach works, but it forces the user to manually track which inner products are implicit, which transposes are adjoints, and how uncertainty propagates through the discretization. When anything non-standard happens — a non-Euclidean norm, a non-uniform grid, a Sobolev prior — the bookkeeping becomes error-prone.

Instead, pygeoinf exposes the following conceptual objects as first-class Python types:

- **Hilbert spaces** (`HilbertSpace`) — carrying their inner product, Riesz maps, and basis structure.
- **Linear operators** (`LinearOperator`) — aware of their domain and codomain, automatically computing the correct Hilbert adjoint from the spaces' Riesz maps.
- **Linear forms** (`LinearForm`) — elements of the dual space, representing individual measurements.
- **Gaussian measures** (`GaussianMeasure`) — priors and posteriors as probability measures on Hilbert spaces.
- **Convex sets and support functions** — geometry of the admissible model set.

Once these objects are defined, a full suite of inversion algorithms — Bayesian, least-squares, minimum-norm, Backus-Gilbert, and deterministic linear inference — work automatically and correctly for any choice of spaces and operators.

1.2 The geophysical inversion problem

The starting point is a physical system described by a *model* m — perhaps the seismic wave-speed field $v_P(x)$, the electrical conductivity $\sigma(x)$, or the subsurface density $\rho(x)$. Some aspects of this

system are measurable: we collect *data* $\tilde{d} = (\tilde{d}_1, \ldots, \tilde{d}_M)$ via instruments (travel-time picks, gravity readings, magnetic measurements, ...). A *forward model* G maps a model to its predicted data:

$$d = Gm + \varepsilon, \tag{1.1}$$

where ε is measurement noise. The *inverse problem* is to determine what equation (1.1) implies about m given the observed data $\tilde{d}$.

All realistic geophysical inverse problems share three features:

1. **Underdetermination.** The data $\tilde{d}$ is a finite-dimensional vector, whereas m is a function (infinite-dimensional in principle). Infinitely many models are consistent with the data exactly; the problem has a non-trivial null space.
2. **Ill-conditioning.** Even the components of m that *are* constrained by the data may be constrained very weakly, amplifying noise in the solution.
3. **Incomplete prior knowledge.** We have partial information about m — physical bounds, smoothness expectations, perhaps a statistical prior — that should inform the inversion.

Any honest inversion framework must confront all three. pygeoinf does so by working at the level of *abstract spaces and operators*, which makes the structure of the problem explicit and prevents silent, basis-dependent errors.

1.3 Why abstract spaces matter

Why this matters

If you are used to working with matrices and vectors in NumPy, you may wonder why pygeoinf insists on Hilbert spaces, dual pairings, and Riesz maps. This section shows you exactly what goes wrong when you ignore these structures — and how pygeoinf gets it right automatically.

To make the argument concrete, consider a simple 1D problem. You want to recover a function $m : [0, 1] \to \mathbb{R}$ from M integral measurements

$$d_i = \int_0^1 g_i(x)\, m(x)\, \mathrm{d}x, \qquad i = 1, \ldots, M. \tag{1.2}$$

You discretize $[0, 1]$ into N cells with centres x_j^* and widths $\Delta x_j > 0$ (possibly non-uniform). Approximate m as piecewise constant: $m(x) \approx m_j$ on cell j, so $\mathbf{m} = (m_1, \ldots, m_N)^\top \in \mathbb{R}^N$. The discrete forward matrix becomes

$$G_{ij} = g_i(x_j^*)\Delta x_j, \qquad \mathbf{d} = \mathbf{G}\mathbf{m} \in \mathbb{R}^M. \tag{1.3}$$

You want the minimum-norm solution: among all $\mathbf{m}$ with $\mathbf{G}\mathbf{m} = \mathbf{d}$, find the one of smallest norm.

The naive approach: use the Euclidean norm on $\mathbb{R}^N$. Minimizing $\|\mathbf{m}\|_2^2 = \sum_j m_j^2$ gives

$$\mathbf{m}^\dagger_{\text{naive}} = \mathbf{G}^\top(\mathbf{G}\mathbf{G}^\top)^{-1}\mathbf{d}. \tag{1.4}$$

The correct approach: use the $L^2([0,1])$ norm. The natural inner product approximating $\|m\|_{L^2}^2 = \int_0^1 m(x)^2 \, \mathrm{d}x$ is

$$(\mathbf{m}, \mathbf{n})_{L^2} \approx \sum_j m_j n_j \Delta x_j = \mathbf{m}^\top \mathbf{M} \mathbf{n}, \qquad \mathbf{M} = \mathrm{diag}(\Delta x_1, \ldots, \Delta x_N). \tag{1.5}$$

Minimizing this gives the qualitatively different solution

$$\mathbf{m}^\dagger = \mathbf{M}^{-1}\mathbf{G}^\top(\mathbf{G}\mathbf{M}^{-1}\mathbf{G}^\top)^{-1}\mathbf{d}. \tag{1.6}$$

Why they differ. If the grid is uniform ($\Delta x_j = 1/N$ for all j), then $\mathbf{M} = \frac{1}{N}\mathbf{I}$ and the two solutions agree up to a constant. But on a non-uniform grid, the naive solution (1.4) *implicitly weights finely-sampled regions more heavily*: minimizing $\sum_j m_j^2$ without the Δx_j weights is not minimizing a physically meaningful norm of m.

The matrix $\mathbf{M}$ is the *Gram matrix* (or mass matrix) of the basis $\{\phi_j\}$ where $\phi_j = \mathbf{1}_{\text{cell } j}/\Delta x_j^{1/2}$. The formula $G^* = \mathbf{M}^{-1}\mathbf{G}^\top$ is the finite-dimensional instance of the abstract relation

$$G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}, \tag{1.7}$$

which pygeoinf enforces automatically once you provide the spaces with their correct inner products.

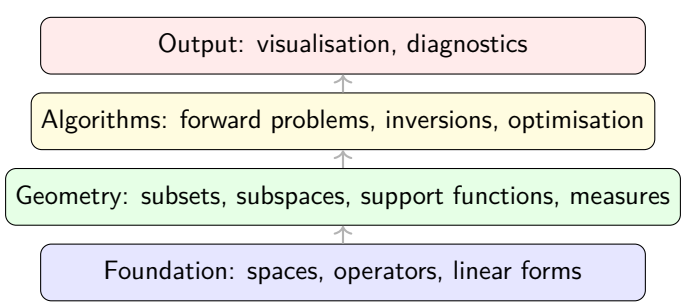
Common Pitfall

The matrix transpose $\mathbf{G}^\top$ is the adjoint only in Euclidean space with the standard dot product. In any other Hilbert space it is not. Using $\mathbf{G}^\top$ instead of $\mathbf{M}_{\mathcal{M}}^{-1}\mathbf{G}^\top\mathbf{M}_{\mathcal{D}}$ produces a solution that is biased toward densely-gridded regions (or, in a Bayesian setting, posterior covariances that are not basis-independent).

This example is the simplest case. As problems become more complex — Sobolev regularity norms, non-trivial data weighting, operators between function spaces — the gap between "naive" and "correct" widens. pygeoinf closes that gap structurally, not by asking the user to remember the right formula each time.

1.4 Package architecture overview

pygeoinf is organized in four ascending layers.



Foundation layer (Part I) The abstract base classes `HilbertSpace`, `LinearOperator`, and `LinearForm` define the mathematical objects. Everything else is built on top of them. No inversion algorithm is aware of the concrete space type; it only calls the abstract interface.

Geometry layer (Part II) Convex subsets (`Ball`, `PolyhedralSet`, ...), subspaces (`AffineSubspace`), support functions (`SupportFunction`), and Gaussian measures (`GaussianMeasure`) all live here. They each accept any `HilbertSpace` and work correctly regardless of its concrete type.

Algorithm layer (Part III) `ForwardProblem`, `LinearBayesianInversion`, `LinearMinimumNormInversion`, `BackusGilbertInversion`, and the convex optimisation algorithms belong here. They compose the foundation and geometry objects into complete solution procedures.

Output layer (Part IV) `SubspaceSlicePlotter` and related tools for visualising results and checking algorithmic correctness.

1.5 How to read this manual

The manual is divided into four parts mirroring the package architecture:

Part I (Foundations) Chapters 2–5. The mathematical vocabulary: spaces, operators, forms. *All users should read at least the motivation sections of these chapters.*

Part II (Geometry) Chapters 6–9. Convex geometry, subspaces, and probabilistic models.

Part III (Problems and Algorithms) Chapters 10–14. How to set up and solve inverse problems.

Part IV (Computational Methods) Chapters 15–16. Numerical backends and visualisation.

Each chapter is self-contained. Forward references are kept to a minimum. The appendices collect a convex analysis primer (Appendix A), functional analysis prerequisites (Appendix B), deferred proofs (Appendix C), and a complete API quick-reference (Appendix D).

Callout boxes.

Yellow — **Why this matters** Motivational content and geophysical perspective.

Blue — **pygeoinf API** The mapping from mathematical concept to Python code.

Green — **Worked Example** Complete mini-examples with both equations and code.

Red — **Common Pitfall** Frequent mistakes and how to avoid them.

1.6 Notation and conventions summary

A complete notation table is given in the Notation and Conventions chapter of the front matter. The most important rules:

- Parentheses $(m_1, m_2)_{\mathcal{M}}$ always denote inner products; angle brackets $\langle \ell, m \rangle$ always denote dual pairings.
- Boldface ($\mathbf{m}$, $\mathbf{G}$) is reserved for finite-dimensional vectors and matrices.
- Calligraphic letters ($\mathcal{M}$, $\mathcal{D}$) denote abstract spaces; Roman capitals (G, T, Π) denote abstract operators.

- G^* always means the *Hilbert* adjoint ($G^* : \mathcal{D} \to \mathcal{M}$); G' means the *Banach* dual ($G' : \mathcal{D}' \to \mathcal{M}'$). They are related by (1.7) but are not the same map.

Chapter 2

Hilbert Spaces

2.1 Why Hilbert spaces?

Why this matters

The inner product on your model space is not a minor technical detail — it determines what "similar models" means, and therefore what your inversion algorithm considers a good solution. Two inversions that differ only in their choice of inner product will in general produce qualitatively different results. pygeoinf makes the inner product an explicit, first-class object so that this choice is always visible and always consistent.

Why not work with plain vectors and matrices? A vector space gives you addition and scalar multiplication, but it has no notion of length or angle. You cannot define a "smallest" element, orthogonality, or projection without more structure. The extra structure needed is an *inner product*.

For geophysical inversion, the inner product on the model space $\mathcal{M}$ is the mathematical encoding of your regularity expectations about the model. A Euclidean inner product treats all degrees of freedom equally, regardless of grid spacing or physical units. An $L^2([0,1])$ inner product weights by area element dx. A Sobolev H^1 inner product additionally penalises gradients. Each choice leads to a different minimum-norm solution; only the "right" one is physically meaningful.

In short: *the inner product is part of the problem specification*, not an implementation detail.

2.2 Definition and axioms

Definition 2.1 (Inner product space)**.** A *real inner product space* is a real vector space H together with a map $(\cdot,\cdot)_H : H \times H \to \mathbb{R}$ satisfying, for all $u, v, w \in H$ and $\alpha \in \mathbb{R}$:

1. **Symmetry:** $(u,v)_H = (v,u)_H$.
2. **Linearity in the first argument:** $(\alpha u + v, w)_H = \alpha(u,w)_H + (v,w)_H$.
3. **Positive definiteness:** $(u,u)_H \geq 0$, with equality only if $u = 0$.

The inner product induces a norm $\|u\|_H := \sqrt{(u,u)_H}$ and hence a metric $d(u,v) = \|u - v\|_H$.

Definition 2.2 (Hilbert space)**.** A real *Hilbert space* is a real inner product space $(H, (\cdot,\cdot)_H)$ that is *complete* with respect to the induced norm: every Cauchy sequence converges to an element of H.

Remark. pygeoinf works exclusively over $\mathbb{R}$. Complex Hilbert spaces are not supported. All spaces are assumed to be *separable*, meaning they admit a countable dense subset; this ensures the existence of countable orthonormal bases.

Orthogonality. Two elements $u, v \in H$ are *orthogonal*, written $u \perp v$, if $(u, v)_H = 0$. A set $S \subset H$ is orthogonal if any two distinct elements of S are orthogonal. If additionally $\|s\|_H = 1$ for all $s \in S$, the set is *orthonormal*.

The parallelogram law. A norm comes from an inner product if and only if it satisfies the *parallelogram law*:

$$\|u+v\|_H^2 + \|u-v\|_H^2 = 2\|u\|_H^2 + 2\|v\|_H^2 \qquad \forall\, u, v \in H. \tag{2.1}$$

The ℓ^p and L^p spaces for $p \neq 2$ are Banach but not Hilbert; they do not satisfy (2.1).

2.3 Inner products, norms, and distances

Proposition 2.3 (Cauchy–Schwarz inequality)**.** *For any $u, v \in H$,*

$$|(u, v)_H| \leq \|u\|_H \, \|v\|_H, \tag{2.2}$$

with equality if and only if u and v are linearly dependent.

The Cauchy–Schwarz inequality guarantees that the cosine of the "angle" between two vectors, $\cos\theta_{u,v} := (u, v)_H/(\|u\|_H\|v\|_H)$, is well-defined and in $[-1, 1]$, even in infinite dimensions.

Polarisation identity. The inner product can be recovered from the norm via

$$(u, v)_H = \tfrac{1}{4}\left(\|u+v\|_H^2 - \|u-v\|_H^2\right). \tag{2.3}$$

This means all metric information about a Hilbert space is encoded in its inner product.

pygeoinf API

`HilbertSpace` exposes:

- `inner_product(x1, x2)`: computes $(x_1, x_2)_H$;
- `norm(x)`: computes $\|x\|_H$;
- `squared_norm(x)`: computes $\|x\|_H^2$ (often cheaper);
- `distance(x1, x2)`: computes $\|x_1 - x_2\|_H$.

Internally, `inner_product` is computed via the dual pairing (see Section 2.4) as $(x_1, x_2)_H = \langle \mathcal{R}^{-1}x_1,\, x_2\rangle$, using `to_dual` and the duality product.

2.4 The dual space and dual pairing

Why this matters

In geophysical terms, a dual vector is exactly what an instrument measures: a number that depends linearly on the model. Understanding the distinction between vectors (models) and dual vectors (measurements) is fundamental to understanding why inversion requires Riesz maps.

Definition 2.4 (Continuous dual space)**.** Let H be a Hilbert space. A *continuous linear functional* on H is a map $\ell : H \to \mathbb{R}$ that is linear ($\ell(\alpha u + v) = \alpha\ell(u) + \ell(v)$) and bounded ($|\ell(u)| \leq C\|u\|_H$ for some $C < \infty$). The collection of all continuous linear functionals on H forms the *(continuous) dual space* H', which is itself a Banach space under the operator norm $\|\ell\|_{H'} := \sup_{\|u\|=1} |\ell(u)|$.

Dual pairing. We denote the evaluation of $\ell \in H'$ at $u \in H$ by the *dual pairing*:

$$\langle \ell, u \rangle := \ell(u). \tag{2.4}$$

We use angle brackets $\langle \cdot, \cdot \rangle$ for dual pairings and parentheses $(\cdot, \cdot)_H$ for inner products throughout this manual; the two are related by the Riesz theorem (Section 2.5).

Two distinct spaces. Elements of H are "primal" vectors (models); elements of H' are "dual" vectors (measurements, functionals, gradients of scalar functions). On a finite-dimensional Euclidean space, $H \cong H'$ via the standard dot product, which is why the distinction is often ignored in numerical practice. On a general Hilbert space — or more acutely, on a Banach space where no inner product exists — the two are different objects and must be handled separately.

Concrete example. Let $H = L^2([0,1])$ and let $g \in L^2([0,1])$. Then the map $\ell : m \mapsto \int_0^1 g(x)m(x)\,\mathrm{d}x$ is a continuous linear functional on H, i.e. $\ell \in H'$. This is precisely the structure of a geophysical measurement: every seismogram, gravity reading, or magnetic datum is a dual vector.

2.5 The Riesz representation theorem

Theorem 2.5 (Riesz representation)**.** *Let H be a real Hilbert space. For every continuous linear functional $\ell \in H'$ there exists a unique $u_\ell \in H$ such that*

$$\ell(v) = (u_\ell,\, v)_H \qquad \forall\, v \in H. \tag{2.5}$$

Moreover, $\|u_\ell\|_H = \|\ell\|_{H'}$. The map $\mathcal{R} : H' \to H$ defined by $\mathcal{R}(\ell) := u_\ell$ is an isometric isomorphism.

Proof sketch. If $\ell = 0$ take $u_\ell = 0$. Otherwise, $\ker \ell$ is a closed proper subspace of H of codimension one. By the projection theorem there exists a unit vector $e \perp \ker \ell$. Set $u_\ell := \ell(e)\, e$. For any $v \in H$, the element $v - \langle \ell, v \rangle e/\ell(e)$ lies in $\ker \ell$, hence is orthogonal to e. A brief calculation shows $(u_\ell, v)_H = \ell(v)$. Uniqueness follows from positive definiteness.

Geometric interpretation. The Riesz representative $u_\ell = \mathcal{R}(\ell)$ is the element of H that "points in the direction" the functional ℓ is measuring. In optimisation this is the *gradient*: if $f(u) = \ell(u)$ then $\nabla f = u_\ell$.

Riesz map and its inverse. We call $\mathcal{R} : H' \to H$ the *Riesz isomorphism* or Riesz map. Its inverse $\mathcal{R}^{-1} : H \to H'$ is defined by $(\mathcal{R}^{-1}u)(v) = (u, v)_H$. Together they establish the identification

$$\langle \ell, v \rangle = (v,\ \mathcal{R}\,\ell)_H \qquad \forall\ \ell \in H',\ v \in H. \tag{2.6}$$

pygeoinf API

The Riesz isomorphism and its inverse are implemented as:

$$\texttt{from_dual} \equiv \mathcal{R} : H' \to H, \qquad \texttt{to_dual} \equiv \mathcal{R}^{-1} : H \to H'.$$

If you need them as `LinearOperator` instances, use the properties `HilbertSpace.riesz` ($\mathcal{R}$) and `HilbertSpace.inverse_riesz` ($\mathcal{R}^{-1}$). The dual pairing evaluates as $\langle \ell, v \rangle = \ell(v)$, implemented by calling the `LinearForm` object directly: `l(v)`.

2.6 Riesz-first design in pygeoinf

The key design choice in `pygeoinf` is that the *primitive* piece of Hilbert-space structure is not an inner-product formula, but the pair of maps implementing the Riesz identification: $\mathcal{R} : H' \to H$ and $\mathcal{R}^{-1} : H \to H'$, exposed in the API as `from_dual` and `to_dual`. Once these are defined, the inner product is *derived* via the duality pairing.

Inner products from duality. Given $u, v \in H$, we define

$$(u, v)_H \;=\; \langle \mathcal{R}^{-1}u, v \rangle \;=\; \langle \texttt{to_dual}(u), v \rangle. \tag{2.7}$$

In code this is the default implementation

```
inner_product(u,v) = duality_product(to_dual(u), v).
```

Consequently, norms and distances are also derived: $\|u\|_H = \sqrt{(u,u)_H}$ and $\|u - v\|_H$ uses the same geometry.

Adjoints depend on the geometry. For a bounded linear operator $A : H_1 \to H_2$, the Hilbert adjoint $A^* : H_2 \to H_1$ is characterised by

$$(Au,\ v)_{H_2} = (u,\ A^*v)_{H_1} \qquad \forall\ u \in H_1,\ v \in H_2. \tag{2.8}$$

Because the inner products on H_1 and H_2 are defined through their Riesz maps, the adjoint is inherently geometry-dependent. In *pygeoinf*, when an operator's adjoint is *derived by the default machinery* (from the dual map and the Riesz identifications), the property `LinearOperator.adjoint` depends on `to_dual`/`from_dual` of the operator's domain and codomain. Changing those maps therefore changes the induced geometry (norms, gradients, and these default adjoints) in a consistent way across the API. If an operator provides an explicit `adjoint_mapping`, that implementation takes precedence and can override the Riesz-derived default.

pygeoinf API

Notation $\leftrightarrow$ `pygeoinf` API (Riesz-first viewpoint):

$$\mathcal{R} \equiv \texttt{HilbertSpace.from_dual}, \qquad \mathcal{R}^{-1} \equiv \texttt{HilbertSpace.to_dual}.$$

$$(u, v)_H \equiv \texttt{HilbertSpace.inner_product(u,v)} \;=\; \langle \texttt{to_dual}(u), v\rangle.$$

$$\texttt{HilbertSpace.riesz} \equiv \mathcal{R}, \qquad \texttt{HilbertSpace.inverse_riesz} \equiv \mathcal{R}^{-1}.$$

Adjoints are geometry-aware by default: when `LinearOperator.adjoint` is derived (rather than explicitly provided), it uses the domain/codomain Riesz maps to satisfy (2.8). An explicitly-provided `adjoint_mapping` overrides this derivation.

2.7 Bases and Representations: The Primal Side

In numerical work we may not always want to manipulate an abstract element $m \in H$ directly. Instead, we might want to work with a sequence of numbers — its *coefficients*. To make this precise, we must choose a basis.

Let H be a Hilbert space with inner product $(\cdot,\cdot)_H$, and let $\{\phi_j\}_{j=1}^{\infty} \subset H$ be a Riesz basis.

By definition, there exist constants $0 < A \le B < \infty$ such that for every finitely supported coefficient sequence $\mathbf{c} \in \ell^2$,

$$A\|\mathbf{c}\|_{\ell^2}^2 \le \left\| \sum_j [\mathbf{c}]_j \phi_j \right\|_H^2 \le B\|\mathbf{c}\|_{\ell^2}^2. \tag{2.9}$$

This inequality has an important interpretation:

- The vector $\sum_j [\mathbf{c}]_j \phi_j$ depends continuously on the coefficients.
- No information is lost or artificially amplified.
- Coefficient vectors and physical vectors measure size in comparable ways.

In other words, the basis is *stable*.

2.7.1 Synthesis: From Coefficients to Vectors

Given a sequence of coefficients $\mathbf{c} = ([\mathbf{c}]_j)$, we construct a vector in H by

$$\pi(\mathbf{c}) := \sum_{j=1}^{\infty} [\mathbf{c}]_j \phi_j. \tag{2.10}$$

This defines the *synthesis operator*

$$\pi : \ell^2 \to H.$$

The Riesz bounds (2.9) imply:

- π is linear,
- π is bounded,
- π is bijective,
- and its inverse is bounded.

Thus π is a topological isomorphism between ℓ^2 and H. This justifies thinking of ℓ^2 as the *coefficient space* associated with the basis $\{\phi_j\}$.

2.7.2 Analysis: From Vectors to Coefficients

Since π is bijective, each vector $m \in H$ has a unique coefficient sequence $\mathbf{c}$ satisfying $\pi(\mathbf{c}) = m$.

This defines the *analysis operator*

$$\Pi := \pi^{-1} : H \to \ell^2.$$

To compute coefficients explicitly, we introduce the *biorthogonal dual basis* $\{\psi_j\} \subset H$, defined by

$$(\phi_j, \psi_k)_H = \delta_{jk}. \tag{2.11}$$

Then the coefficients of m are given by

$$\Pi(m) = \big[(m, \psi_j)_H\big]_j. \tag{2.12}$$

Thus:

$$\pi \circ \Pi = \mathrm{Id}_H, \qquad \Pi \circ \pi = \mathrm{Id}_{\ell^2}.$$

We now have a complete correspondence:

$$\text{coefficients} \quad \longleftrightarrow \quad \text{vectors in } H.$$

pygeoinf API

Notation ↔ pygeoinf (analysis/synthesis). Fix a concrete `HilbertSpace` instance `H`. Its finite working dimension is `H.dim` (this is where "N" enters in *pygeoinf*). Then:

- The *primary coefficient route* $\Pi : H \to \ell^2$ is exposed as `H.coordinate_projection` (a `LinearOperator` wrapping `to_components`).
- The synthesis map $\pi : \ell^2 \to H$ is exposed as `H.coordinate_inclusion` (wrapping `from_components`).
- The "inner-product vector" route uses the adjoint $\pi^{*_{\ell^2}} : H \to \ell^2$, available as `H.coordinate_inclusion.adj`

In short, the coefficient vector is `c = H.coordinate_projection(m)` (this is Πm), and reconstruction is `m = H.coordinate_inclusion(c)` (this is $\pi\mathbf{c}$).

2.8 The Gram Matrix: The Metric Hidden in the Basis

Up to now we have only discussed linear representation. We now ask a deeper question:

How is the inner product of H expressed in terms of coefficients?

Let $\mathbf{c} = \Pi m$ and $\mathbf{d} = \Pi n$. Then

$$(m, n)_H = (\pi\mathbf{c}, \pi\mathbf{d})_H.$$

If the basis were orthonormal, this would equal $(\mathbf{c}, \mathbf{d})_{\ell^2}$. But for a general Riesz basis this is false.

2.8.1 The Induced Coefficient Inner Product

Define

$$(\mathbf{c}, \mathbf{d})_\pi := (\pi\mathbf{c}, \pi\mathbf{d})_H. \tag{2.13}$$

This turns ℓ^2 into a Hilbert space in which

$$\pi : (\ell^2, (\cdot,\cdot)_\pi) \to (H, (\cdot,\cdot)_H)$$

is an isometry.

Thus the geometry of H is transferred to coefficient space.

2.8.2 The Gram Operator

We now express this induced inner product relative to the *standard* ℓ^2 inner product

$$(\mathbf{c}, \mathbf{d})_{\ell^2} = \sum_j [\mathbf{c}]_j [\mathbf{d}]_j.$$

There exists a unique bounded, self-adjoint, strictly positive operator

$$\mathbf{M}_\phi : \ell^2 \to \ell^2$$

such that

$$(\mathbf{c}, \mathbf{d})_\pi = (\mathbf{M}_\phi \mathbf{c}, \mathbf{d})_{\ell^2}. \tag{2.14}$$

A direct calculation shows

$$[\mathbf{M}_\phi]_{jk} = (\phi_k, \phi_j)_H, \qquad \mathbf{M}_\phi = \pi^{*_{\ell^2}} \pi. \tag{2.15}$$

where $*_{\ell^2}$ denotes adjoint w.r.t. the ℓ^2 inner product. The matrix $\mathbf{M}_\phi$ measures how the basis vectors overlap. It is the *metric tensor* of the representation.

2.8.3 Why the Gram Matrix Matters

The Riesz basis bounds imply

$$A\|\mathbf{c}\|_{\ell^2}^2 \le (\mathbf{M}_\phi \mathbf{c}, \mathbf{c})_{\ell^2} \le B\|\mathbf{c}\|_{\ell^2}^2,$$

so $\mathbf{M}_\phi$ is symmetric positive definite and invertible.

Moreover, the analysis operator can be computed via

$$\boxed{\Pi = \mathbf{M}_\phi^{-1} \pi^{*_{\ell^2}}.} \tag{2.16}$$

This identity is fundamental:

> *We do not need the dual basis explicitly. The Gram matrix contains all the necessary information.*

pygeoinf API

Computing coefficients via the Gram/mass operator in code. When you know the synthesis map π and can form its Hilbert adjoint $\pi^{*_{\ell^2}}$, you can recover the analysis operator via $\Pi = \mathbf{M}_\phi^{-1}\pi^{*_{\ell^2}}$ with $\mathbf{M}_\phi = \pi^{*_{\ell^2}}\pi$.

In *pygeoinf*, with `H: HilbertSpace` and `N = H.dim`:

- `pi = H.coordinate_inclusion` is $\pi : \mathbb{R}^N \to H$,
- `pi_star = pi.adjoint` is $\pi^{*_{\ell^2}} : H \to \mathbb{R}^N$,
- `M = pi_star @ pi` is $\mathbf{M}_\phi : \mathbb{R}^N \to \mathbb{R}^N$.

Then for a vector $m \in H$:

1. Form `b = pi_star(m)` (this is the $\pi^{*_{\ell^2}} m$ "inner-product vector"),
2. Solve `M c = b` for `c` (this is Πm).

Practical note: use `.adjoint` for Hilbert adjoints; `.dual` is the Banach dual (π') and differs unless you are in a standard Euclidean setting.

2.9 Duality and the Riesz Map in Coordinates

The dual space H' contains functionals. The Riesz isomorphism

$$\mathcal{R}_H : H' \to H$$

identifies each functional with its representing vector.

Its inverse

$$\mathcal{R}_H^{-1} : H \to H'$$

turns a vector into its corresponding functional.

2.9.1 What Happens in Coordinates

Suppose $m \in H$ has coefficient vector $\mathbf{c} = \Pi m \in \ell^2$. Equivalently, $m = \pi\mathbf{c}$.

We would like to understand the *coordinate representation* of the functional $\mathcal{R}_H^{-1} m \in H'$. Concretely, we ask:

If we turn m into a functional via $\mathcal{R}_H^{-1}$, and then extract its coefficients in coefficient space, what do we get?

Recall:

- The inverse Riesz map $\mathcal{R}_H^{-1} : H \to H'$ is characterized by

$$\langle \mathcal{R}_H^{-1} u, v\rangle = (u, v)_H \qquad \forall\, u, v \in H. \tag{2.17}$$

- The Banach adjoint (dual operator) of synthesis $\pi : \ell^2 \to H$ is

$$\pi' : H' \to (\ell^2)', \qquad (\pi'\ell)(\mathbf{d}) := \langle \ell, \pi\mathbf{d}\rangle \qquad \forall\, \ell \in H',\ \mathbf{d} \in \ell^2. \tag{2.18}$$

- The coefficient-space Riesz map $\mathcal{R}_{\ell^2} : (\ell^2)' \to \ell^2$ (for the *standard* ℓ^2 inner product) is characterized by
$$(\mathcal{R}_{\ell^2}\eta, \mathbf{d})_{\ell^2} = \eta(\mathbf{d}) \qquad \forall\, \eta \in (\ell^2)',\ \mathbf{d} \in \ell^2. \tag{2.19}$$

- The Gram/mass operator $\mathbf{M}_\phi : \ell^2 \to \ell^2$ is defined by
$$(\pi\mathbf{c}, \pi\mathbf{d})_H = (\mathbf{M}_\phi\mathbf{c}, \mathbf{d})_{\ell^2} \qquad \forall\, \mathbf{c}, \mathbf{d} \in \ell^2. \tag{2.20}$$
Equivalently, $\mathbf{M}_\phi = \pi^{*_{\ell^2}}\pi$.

Derivation of the coordinate identity. Let $m \in H$ be arbitrary and set $\mathbf{c} := \Pi m$ so that $m = \pi\mathbf{c}$. Define the coefficient functional
$$\eta := \pi'(\mathcal{R}_H^{-1}m) \in (\ell^2)'.$$
We compute $\eta(\mathbf{d})$ for an arbitrary coefficient vector $\mathbf{d} \in \ell^2$:
$$\begin{aligned}
\eta(\mathbf{d}) &= (\pi'(\mathcal{R}_H^{-1}m))(\mathbf{d}) && \text{(definition of } \eta) \\
&= \left\langle \mathcal{R}_H^{-1}m,\ \pi\mathbf{d} \right\rangle && \text{(definition of } \pi') && (2.21)\\
&= (m, \pi\mathbf{d})_H && \text{(Riesz characterization (2.17))} && (2.22)\\
&= (\pi\mathbf{c}, \pi\mathbf{d})_H && \text{(since } m = \pi\mathbf{c}) && (2.23)\\
&= (\mathbf{M}_\phi\mathbf{c}, \mathbf{d})_{\ell^2} && \text{(Gram identity (2.20)).} && (2.24)
\end{aligned}$$
Now compare (2.24) with the defining property of the coefficient-space Riesz map (2.19). Since (2.19) says that $\mathcal{R}_{\ell^2}\eta$ is the unique vector in ℓ^2 satisfying
$$(\mathcal{R}_{\ell^2}\eta, \mathbf{d})_{\ell^2} = \eta(\mathbf{d}) \quad \forall\, \mathbf{d},$$
and we have just shown that $\eta(\mathbf{d}) = (\mathbf{M}_\phi\mathbf{c}, \mathbf{d})_{\ell^2}$ for all $\mathbf{d}$, it follows that
$$\mathcal{R}_{\ell^2}\eta = \mathbf{M}_\phi\mathbf{c}.$$
Substituting back $\eta = \pi'(\mathcal{R}_H^{-1}m)$ and $\mathbf{c} = \Pi m$ yields the operator identity
$$\boxed{\mathcal{R}_{\ell^2}\,\pi'\,\mathcal{R}_H^{-1} = \mathbf{M}_\phi\,\Pi.} \tag{2.25}$$

Interpretation: why this means $\mathbf{M}_\phi$ is the coordinate representation of $\mathcal{R}_H^{-1}$. Equation (2.25) is an equality of maps $H \to \ell^2$. Read it from right to left and from left to right:

- The map $\Pi : H \to \ell^2$ takes a vector m and returns its *primal* coefficient vector $\mathbf{c} = \Pi m$.
- Multiplication by $\mathbf{M}_\phi$ converts this primal coefficient vector into $\mathbf{M}_\phi\mathbf{c} \in \ell^2$.
- The composite $\mathcal{R}_{\ell^2}\,\pi'\,\mathcal{R}_H^{-1}$ does the same job but via the dual space: it first turns m into its functional $\mathcal{R}_H^{-1}m \in H'$, then extracts the corresponding coefficient functional $\pi'(\mathcal{R}_H^{-1}m) \in (\ell^2)'$, and finally identifies that functional with a coefficient vector in ℓ^2 using $\mathcal{R}_{\ell^2}$.

Thus, after choosing coordinates via Π (on H) and via $\mathcal{R}_{\ell^2} \circ \pi'$ (on H'), the inverse Riesz map $\mathcal{R}_H^{-1} : H \to H'$ is represented by the operator $\mathbf{M}_\phi$ on the coefficient space. In this precise sense,

$\mathbf{M}_\phi$ is the coordinate (matrix/operator) representation of $\mathcal{R}_H^{-1}$.

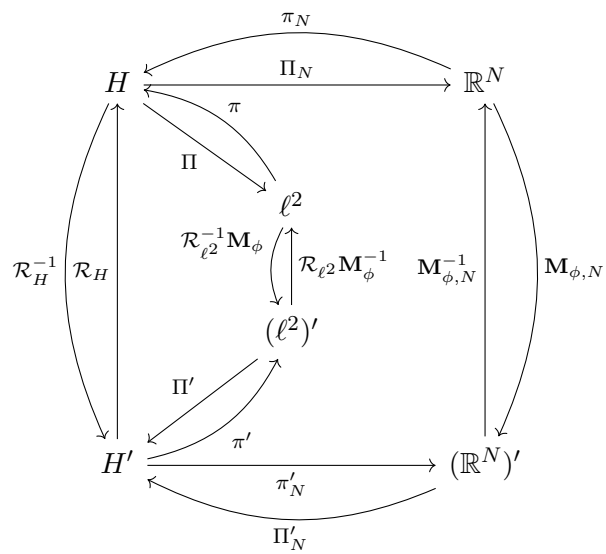


Figure 2.1: Commutative diagram of primal and dual coefficient maps in *pygeoinf*. The analysis operator $\Pi : H \to \ell^2$ extracts primal coefficients, while its inverse (synthesis) $\pi : \ell^2 \to H$ reconstructs vectors . The Riesz isomorphism $\mathcal{R}_H : H' \to H$ identifies dual functionals with primal representatives; its inverse $\mathcal{R}_H^{-1} : H \to H'$ maps a vector to its unique functional form $\langle \mathcal{R}_H^{-1} u, v\rangle = (u, v)_H$. In the coefficient space, the Gram matrix $\mathbf{M}_\phi$ acts as the coordinate representation of the inverse Riesz map, transforming primal coefficients into dual coefficients . The diagram commutes such that extracting dual coefficients directly from a functional is equivalent to mapping primal coefficients through the Gram matrix: $\mathcal{R}_{\ell^2} \circ \pi' \circ \mathcal{R}_H^{-1} = \mathbf{M}_\phi \circ \Pi$ (identifying $(\ell^2)' \cong \ell^2$ via $\mathcal{R}_{\ell^2}$).

Equivalently, one may solve (2.25) for $\mathcal{R}_H^{-1}$:

$$\mathcal{R}_H^{-1} = (\pi')^{-1}\, \mathcal{R}_{\ell^2}^{-1}\, \mathbf{M}_\phi\, \Pi, \tag{2.26}$$

which makes explicit that $\mathbf{M}_\phi$ is the "middle" coordinate map that carries primal coefficients to dual coefficients.

2.10 Finite-Dimensional Approximation

In practice we never work with infinite coefficient sequences. Instead, we truncate the basis to the first N elements and work in the finite-dimensional subspace

$$H_N := \operatorname{span}\{\phi_1, \dots, \phi_N\} \subset H. \tag{2.27}$$

Everything in the infinite-dimensional theory now reduces to linear algebra.

2.10.1 Finite Synthesis and Analysis

Define the truncated synthesis operator

$$\pi_N : \mathbb{R}^N \to H_N, \qquad \pi_N(\mathbf{c}) := \sum_{j=1}^{N} [\mathbf{c}]_j \phi_j. \tag{2.28}$$

Since $\{\phi_1, \ldots, \phi_N\}$ is linearly independent, π_N is a linear isomorphism.

Its inverse is the truncated analysis operator

$$\Pi_N : H_N \to \mathbb{R}^N, \qquad \Pi_N(m) = \mathbf{c} \quad \text{such that } m = \pi_N(\mathbf{c}). \tag{2.29}$$

Thus

$$\pi_N \circ \Pi_N = \mathrm{Id}_{H_N}, \qquad \Pi_N \circ \pi_N = \mathrm{Id}_{\mathbb{R}^N}. \tag{2.30}$$

2.10.2 The Finite Gram Matrix

The geometry of H_N is encoded by the $N \times N$ Gram (mass) matrix

$$[\mathbf{M}_{\phi,N}]_{jk} := (\phi_k, \phi_j)_H, \qquad 1 \le j, k \le N. \tag{2.31}$$

A direct calculation shows

$$(\pi_N \mathbf{c}, \pi_N \mathbf{d})_H = (\mathbf{c}, \mathbf{M}_{\phi,N}\mathbf{d})_{\mathbb{R}^N}, \tag{2.32}$$

where $(\mathbf{c}, \mathbf{d})_{\mathbb{R}^N} = \mathbf{c}^\top \mathbf{d}$ is the standard Euclidean inner product.

Thus $M_{\phi,N}$ represents the induced inner product

$$(\mathbf{c}, \mathbf{d})_\pi := (\pi_N \mathbf{c}, \pi_N \mathbf{d})_H.$$

2.10.3 Properties of the Finite Gram Matrix

The matrix $\mathbf{M}_{\phi,N}$ is:

- symmetric,
- strictly positive definite,
- invertible.

Indeed,

$$\mathbf{c}^\top \mathbf{M}_{\phi,N}\mathbf{c} = \|\pi_N \mathbf{c}\|_H^2,$$

and this vanishes only if $\mathbf{c} = 0$.

2.10.4 Finite Analysis via the Gram Matrix

Taking adjoints relative to the standard Euclidean inner product,

$$\pi_N^* : H_N \to \mathbb{R}^N, \qquad (\pi_N^* m)_j = (m, \phi_j)_H.$$

The truncated analysis operator satisfies

$$\boxed{\Pi_N = \mathbf{M}_{\phi,N}^{-1} \pi_N^*.} \tag{2.33}$$

Thus coefficients can be computed without explicitly constructing the dual basis:

1. Compute inner products with ϕ_j.
2. Solve a linear system with matrix $\mathbf{M}_{\phi,N}$.

2.10.5 Finite Riesz Map in Coordinates

The Riesz isomorphism restricted to H_N satisfies

$$\langle \mathcal{R}_H^{-1} m, v \rangle = (m, v)_H, \qquad m, v \in H_N.$$

In coefficient form, let $\mathbf{c} = \Pi_N m$. Then the dual coefficients of $\mathcal{R}_H^{-1} m$ are

$$\boxed{\mathcal{R}_{\mathbb{R}^N}\, \pi'_N\, \mathcal{R}_H^{-1} = \mathbf{M}_{\phi,N}\, \Pi_N.} \tag{2.34}$$

In purely matrix terms:

$$\text{dual coefficients} = \mathbf{M}_{\phi,N}\,(\text{primal coefficients}). \tag{2.35}$$

Thus the finite Gram matrix is the matrix representation of the inverse Riesz map on H_N.

2.10.6 Summary: What Changes in Finite Dimensions

In infinite dimensions:

- ℓ^2 and H are infinite Hilbert spaces.
- $\mathbf{M}_\phi$ is a bounded positive operator.
- Riesz maps are abstract isomorphisms.

In finite dimensions:

- All spaces are Euclidean.
- All operators are matrices.
- The Riesz map becomes multiplication by a symmetric positive definite matrix.
- The Gram matrix *is* that matrix.

Thus the finite-dimensional setting is not an approximation of the structure — it is the exact same structure realized in linear algebra form.

Why this matters

This is the key theory $\leftrightarrow$ computation bridge: the abstract operators (π, Π, adjoints, and Riesz maps) do not disappear when you discretize. Choosing a concrete `HilbertSpace` (and therefore a specific `dim`) *is* the discretization. Once that choice is made, *pygeoinf* computes adjoints using the declared inner products, so the same identities (e.g. $\Pi = \mathbf{M}^{-1}\pi^{*_{\ell^2}}$) remain valid and numerically meaningful. As you increase `dim` (or refine the underlying space), you are moving along a controlled family of finite problems that approximate the intended infinite-dimensional model.

2.11 Concrete spaces in pygeoinf

This section introduces the built-in Hilbert space implementations and explains how to create your own.

2.11.1 `EuclideanSpace`

$\mathbb{R}^n$ with the standard inner product $(u, v) = \sum_i u_i v_i = \mathbf{u}^\top \mathbf{v}$. The Riesz map is the identity: $\mathcal{R} = \mathrm{Id}$. The basis $\{\phi_j\} = \{\mathbf{e}_j\}$ is the standard ONB, so $\mathbf{M}_\phi = \mathbf{I}$ and $\Pi = \pi^{*\ell^2} = \mathrm{Id}$.

When you use `EuclideanSpace(n)`, adjacent operator transposes *are* adjoints, and you recover the classical formulas of linear algebra. This is the appropriate choice when variables represent coefficients in a pre-orthonormalised basis (e.g. after a change of variables $\tilde{m} = \mathbf{L}m$ where $\mathbf{L}$ is a Cholesky factor of $\mathbf{M}_\phi$).

2.11.2 `MassWeightedHilbertSpace`

Given a Hilbert space H and a bounded symmetric positive operator $\mathbf{W} : H \to H'$ (the "weight operator"), the *mass-weighted space* $H_{\mathbf{W}}$ is the same underlying vector space but with inner product

$$(u, v)_{H_{\mathbf{W}}} = \langle \mathbf{W}u,\, v \rangle. \tag{2.36}$$

The Riesz maps become $\mathcal{R}_{\mathbf{W}} = \mathcal{R} \circ \mathbf{W}$ and $\mathcal{R}_{\mathbf{W}}^{-1} = \mathbf{W}^{-1} \circ \mathcal{R}^{-1}$.

Motivating use case: discretized L^2. Using the non-uniform grid inner product from Section 1.3, we set $H = \mathbb{R}^N$, $H_{\mathbf{W}} = \mathbb{R}^N$ with weight $\mathbf{W} = \mathbf{M} = \mathrm{diag}(\Delta x_j)$. Then $(u, v)_{H_{\mathbf{W}}} = \sum_j u_j v_j \Delta x_j \approx \int_0^1 u(x)v(x)\,\mathrm{d}x$.

2.11.3 `HilbertSpaceDirectSum`

Given Hilbert spaces H_1 and H_2, their *Hilbert direct sum* $H_1 \oplus H_2$ consists of pairs (u_1, u_2) with

$$\big((u_1, u_2),\, (v_1, v_2)\big)_{H_1 \oplus H_2} = (u_1, v_1)_{H_1} + (u_2, v_2)_{H_2}. \tag{2.37}$$

The Riesz maps and analysis/synthesis operators decompose blockwise. This is the natural structure for models that are concatenations of separately-structured fields (e.g. P-wave and S-wave velocity perturbations).

2.11.4 Implementing your own `HilbertSpace`

To encode a non-standard model space, subclass `HilbertSpace`. The abstract interface requires exactly eight methods:

pygeoinf API

Subclass `HilbertSpace` and implement:

- `dim` — integer dimension N of the space.
- `zero()` — return the zero element.
- `basis_vector(j)` — return ϕ_j for $j = 0, \ldots, N-1$.
- `inner_product(x1, x2)` — compute $(x_1, x_2)_H$.
- `to_dual(x)` — compute $\mathcal{R}^{-1}(x) \in H'$.
- `from_dual(xp)` — compute $\mathcal{R}(\ell) \in H$ for $\ell \in H'$.
- `to_components(x)` — compute $\Pi(x) \in \mathbb{R}^N$.

- `from_components(c)` — compute $\pi(\mathbf{c}) \in H$.

All other methods (`norm`, `squared_norm`, `distance`, `riesz`, `inverse_riesz`, `coordinate_projection`, `coordinate_inclusion`, ...) are derived automatically from the above.

Worked Example

Example (quadrature-weighted L^2 via `MassWeightedHilbertSpace`). Suppose you discretize $[0, 1]$ on a non-uniform grid with cell widths Δx_j and represent a function by its sampled values $\mathbf{u} = (u_j) \in \mathbb{R}^N$. The quadrature-weighted inner product

$$(\mathbf{u}, \mathbf{v}) \approx \sum_{j=1}^{N} u_j v_j \, \Delta x_j$$

is obtained by taking an underlying Euclidean space $X = \mathbb{R}^N$ and a diagonal mass operator $\mathbf{M} = \mathrm{diag}(\Delta x_j)$, then forming the mass-weighted space $X_{\mathbf{M}}$.

In *pygeoinf* this is the canonical pattern:

- `X = EuclideanSpace(N)`
- `dx = np.asarray([...])  # quadrature weights (e.g.  cell widths)`
- `M = LinearOperator.from_matrix(X, X, np.diag(dx))`
- `Minv = LinearOperator.from_matrix(X, X, np.diag(1.0/dx))`
- `H = MassWeightedHilbertSpace(X, M, Minv)`

Then `H.inner_product(u, v)` computes $\langle \mathbf{M}\mathbf{u}, \mathbf{v} \rangle$ and the Riesz maps become $\mathcal{R}_H = \mathcal{R}_X \circ \mathbf{M}$ and $\mathcal{R}_H^{-1} = \mathbf{M}^{-1} \circ \mathcal{R}_X^{-1}$, exactly matching (2.36).

Chapter 3

Linear Forms

3.1 Motivation: measurements as linear functionals

Many inverse problems can be written (after linearization, if needed) as

$$\tilde{d}_i = \ell_i(m) + \varepsilon_i, \qquad i = 1, \ldots, M,$$

where $m \in \mathcal{M}$ is the model, each $\ell_i \in \mathcal{M}'$ is a continuous linear functional, and ε_i is measurement noise. The data vector is therefore best understood as a *stack of linear forms* acting on the model.

Worked Example

Example (a localized average). Let $\mathcal{M} = L^2(\Omega)$ and let $w \in L^2(\Omega)$ be a fixed weighting function describing an instrument footprint or averaging kernel. Then

$$\ell(m) := \int_\Omega w(x)\, m(x)\, \mathrm{d}x$$

is a continuous linear form. In a discretization, the only question is how m is represented and which inner product defines the geometry of $\mathcal{M}$.

Why this matters

Every single datum in your data vector is the result of applying a linear functional to the model. Understanding linear forms is understanding what your data actually *are* mathematically.

3.2 Definition

Let $\mathcal{M}$ be a (real) Hilbert space and let $\mathcal{M}'$ denote its continuous dual. A *linear form* is a continuous linear map

$$\ell : \mathcal{M} \to \mathbb{R}.$$

We write the duality pairing as

$$\langle \ell, m \rangle := \ell(m), \qquad \ell \in \mathcal{M}',\ m \in \mathcal{M}.$$

In *pygeoinf*, a linear form is represented by the class `LinearForm(domain, ...)`, where `domain` is the Hilbert space on which the form acts.

3.3 Representation in coordinates

The library uses the domain's coordinate maps $\Pi : \mathcal{M} \to \mathbb{R}^N$ (`to_components`) and $\pi : \mathbb{R}^N \to \mathcal{M}$ (`from_components`) to provide a computational coordinate system. This is a *chosen representation*: it need not be orthonormal.

Let $\phi_j := \pi(e_j)$ denote the corresponding computational basis vectors (in code: `domain.basis_vector(j)`), where e_j are the standard unit vectors in $\mathbb{R}^N$. The component vector of a linear form ℓ is defined as

$$\mathbf{c}_\ell := (\ell(\phi_0), \ldots, \ell(\phi_{N-1}))^\top \in \mathbb{R}^N.$$

Then for any $m \in \mathcal{M}$ with coefficient vector $\mathbf{m} := \Pi m$, linearity gives

$$\ell(m) = \sum_{j=0}^{N-1} \ell(\phi_j)\, \mathbf{m}_j = \mathbf{c}_\ell^\top\, \mathbf{m}.$$

This is exactly the evaluation rule used by `LinearForm`: `l(m) = dot(l.components, domain.to_components(m))`.

Two equivalent construction paths in the API

pygeoinf API

(1) Provide components directly.
`l = LinearForm(domain, components=c)`
Here `c` is a 1D array of length `domain.dim` representing $\mathbf{c}_\ell$.

(2) Provide a callable mapping and let pygeoinf compute components.
`l = LinearForm(domain, mapping=f, parallel=True, n_jobs=-1)`
The library computes `l.components[i]` by evaluating `f` on the computational basis vectors ϕ_i = `domain.basis_vector(i)` (equivalently ϕ_i = `domain.from_components`(e_i)): $\mathbf{c}_{\ell,i} = f(\phi_i)$. When `parallel=True`, this is done with a joblib backend; `n_jobs` controls the number of workers.

Common Pitfall

The `mapping` construction is convenient, but it assumes the callable really is linear (and deterministic) on the domain. If your `mapping` includes hidden nonlinearities (thresholding, clipping, normalization, stateful caches), the computed components will be inconsistent with the linear algebra and subsequent results may be meaningless.

3.4 The Riesz representative (gradient)

Because $\mathcal{M}$ is a Hilbert space, every $\ell \in \mathcal{M}'$ has a unique *Riesz representative* $g_\ell \in \mathcal{M}$ such that

$$\ell(m) = \langle \ell, m \rangle = (g_\ell, m)_{\mathcal{M}} \qquad \text{for all } m \in \mathcal{M}.$$

Equivalently, $g_\ell = \mathcal{R}_{\mathcal{M}}(\ell)$, where $\mathcal{R}_{\mathcal{M}} : \mathcal{M}' \to \mathcal{M}$ is the Riesz isomorphism listed in the notation table (`from_dual`).

In *pygeoinf*, this is the mechanism that injects *geometry* into linear forms: the component vector $\mathbf{c}_\ell$ encodes the action of ℓ on the chosen coordinate system, while the inner product on $\mathcal{M}$ determines the corresponding gradient vector. Concretely,

$$\nabla \ell \equiv g_\ell = \texttt{domain.from_dual(l)}.$$

The Hessian of any linear form is the zero operator.

Remark. If the coordinate system is not orthonormal, then the coefficient vector Πg_ℓ is generally *not* equal to $\mathbf{c}_\ell$. They are related by the Gram/mass matrix of the chosen basis. This is why `from_dual` (Riesz) must be used to obtain the correct gradient in the domain geometry.

3.5 Interoperability: viewing a linear form as an operator

Some parts of the library (and many external numerical tools) are written in terms of operators rather than functionals. For interoperability, `LinearForm` exposes an adapter `as_linear_operator` that views ℓ as an operator $\mathcal{M} \to \mathbb{R}$ by returning a `LinearOperator` with codomain `EuclideanSpace(1)`.

This should be understood purely as an interface bridge, not as a conceptual re-framing of linear forms. In particular, we do *not* treat a linear form as a "rank-1 operator" in the theory: its natural home is the dual space $\mathcal{M}'$.

Implementing your own linear form (practical guidance)

Most users should not subclass anything: instantiate `LinearForm` using one of the two construction paths above. To ensure correctness, keep the following invariants in mind:

- **Component meaning.** The stored array `l.components` is $\mathbf{c}_\ell$ with entries $\mathbf{c}_{\ell,i} = \ell(\phi_i)$ for the domain's chosen basis vectors $\phi_i =$ `domain.basis_vector(i)`.
- **Evaluation rule.** Evaluation uses the coordinate projection: $\ell(m) = \mathbf{c}_\ell^\top \Pi m$. If you change how `to_components/from_components` represent the domain, you also change what `components` mean.
- **Geometry lives in the Riesz map.** Use `domain.from_dual(l)` to obtain the gradient/Riesz representative.

Convenience methods commonly used in implementations and experiments:

- Arithmetic: `l1 + l2`, `l1 - l2`, `a * l`, `l / a`, and in-place `l *= a`, `l += other`.
- `l.copy()` for a deep copy.
- `l.as_linear_operator` to plug a form into operator-based code.
- `LinearForm.from_linear_operator(op)` to wrap an operator `op: domain -> EuclideanSpace(1)` as a linear form.

pygeoinf API

`LinearForm(domain, components=...)` stores $\mathbf{c}_\ell$ and evaluates via `dot(components, domain.to_components`
`LinearForm(domain, mapping=..., parallel=..., n_jobs=...)` computes components by evaluating the mapping on `domain.basis_vector(i)`.

Useful methods/properties: `__call__(x)`, `components`, `copy()`, `as_linear_operator`, `from_linear_operator(`

Chapter 4

Linear Operators

4.1 Motivation: the forward operator

In inverse problems we distinguish a *model space* $\mathcal{M}$ and a *data space* $\mathcal{D}$. The forward physics is encoded as a linear map $G : \mathcal{M} \to \mathcal{D}$ taking a model m to the data $d = G(m)$ it predicts. Throughout this chapter we use the evaluation convention $G(x)$ (rather than juxtaposition Gx) to emphasize that operators are functions, and to match the callable operator objects in the `pygeoinf` API.

Why this matters

The forward operator G is the mathematical encoding of your physics. Getting its adjoint right is critical for every inversion algorithm — and the adjoint depends on the inner products of both spaces, not just the matrix transpose.

4.2 Definition and properties

Definition (bounded linear operator). Let $\mathcal{M}$ and $\mathcal{D}$ be normed vector spaces (in particular, Hilbert spaces in the applications in this manual). A map $G : \mathcal{M} \to \mathcal{D}$ is *linear* if

$$G(\alpha m_1 + \beta m_2) = \alpha\, G(m_1) + \beta\, G(m_2) \qquad \forall\, m_1, m_2 \in \mathcal{M},\ \alpha, \beta \in \mathbb{R}. \tag{4.1}$$

It is *bounded* if there exists a constant $C < \infty$ such that

$$\|G(m)\|_{\mathcal{D}} \le C\, \|m\|_{\mathcal{M}} \qquad \forall\, m \in \mathcal{M}. \tag{4.2}$$

The smallest such constant is the *operator norm* $\|G\| = \sup_{\|m\|_{\mathcal{M}}=1} \|G(m)\|_{\mathcal{D}}$. In Hilbert spaces, boundedness is equivalent to continuity, and it is the assumption that makes adjoints and discretisations behave well.

pygeoinf API

Notation $\leftrightarrow$ `pygeoinf` API (linear operators):

$G : \mathcal{M} \to \mathcal{D}$ $\equiv$ `LinearOperator(domain, codomain, mapping)` and evaluate via `G(m)`.

G' (Banach dual) $\equiv$ `G.dual`, G^* (Hilbert adjoint) $\equiv$ `G.adjoint`.

$\mathcal{R}_{\mathcal{M}} \equiv$ `domain.from_dual`, $\mathcal{R}_{\mathcal{M}}^{-1} \equiv$ `domain.to_dual`, $\mathcal{R}_{\mathcal{D}} \equiv$ `codomain.from_dual`, $\mathcal{R}_{\mathcal{D}}^{-1} \equiv$ `codomain.to`

4.3 The Banach dual (transpose) G'

The definition of G' requires only dual pairings; it makes sense for any bounded linear operator between normed spaces.

Typing. Let $\mathcal{M}$ and $\mathcal{D}$ be normed vector spaces and let $G : \mathcal{M} \to \mathcal{D}$ be a bounded linear operator. Then the continuous dual spaces $\mathcal{M}'$ and $\mathcal{D}'$ are Banach spaces (Definition 2.4), and the dual pairing is denoted by $\langle \cdot, \cdot \rangle$.

Definition (Banach dual / transpose). The *Banach dual* (also called the *transpose*) of G is the bounded linear map

$$G' : \mathcal{D}' \to \mathcal{M}' \tag{4.3}$$

defined by

$$\langle G'(d'), m \rangle := \langle d', G(m) \rangle \qquad \forall\, d' \in \mathcal{D}',\ m \in \mathcal{M}. \tag{4.4}$$

Equivalently, for each $d' \in \mathcal{D}'$, the element $G'(d') \in \mathcal{M}'$ is the functional

$$(G'(d'))(m) = d'(G(m)). \tag{4.5}$$

Boundedness. From (4.4) and (4.2), for any $d' \in \mathcal{D}'$ we have

$$\|G'(d')\|_{\mathcal{M}'} = \sup_{\|m\|_{\mathcal{M}}=1} \big|\langle G'(d'), m \rangle\big| = \sup_{\|m\|_{\mathcal{M}}=1} \big|\langle d', G(m) \rangle\big| \le \|d'\|_{\mathcal{D}'}\, \|G\|, \tag{4.6}$$

so G' is bounded with $\|G'\| \le \|G\|$.

Why G' is not an adjoint. The map G' acts on *dual* elements and produces a dual element. Even when $\mathcal{M}$ and $\mathcal{D}$ are Hilbert spaces, the Hilbert adjoint G^* (next section) is a map $\mathcal{D} \to \mathcal{M}$ between the primal spaces. To pass from G' to G^* we must use the Riesz maps, i.e. identify dual vectors with their Riesz representatives in the primal spaces.

4.4 The Hilbert adjoint G^*

From here on we assume that both spaces are Hilbert spaces, so that we have inner products and Riesz maps available.

Typing. Let $\mathcal{M}$ and $\mathcal{D}$ be Hilbert spaces with inner products $(\cdot, \cdot)_{\mathcal{M}}$ and $(\cdot, \cdot)_{\mathcal{D}}$, and let $G : \mathcal{M} \to \mathcal{D}$ be bounded and linear.

Definition (Hilbert adjoint). The *Hilbert adjoint* of G is the unique bounded linear operator

$$G^* : \mathcal{D} \to \mathcal{M} \tag{4.7}$$

such that

$$(G(m), d)_{\mathcal{D}} = (m, G^*(d))_{\mathcal{M}} \qquad \forall\, m \in \mathcal{M},\ d \in \mathcal{D}. \tag{4.8}$$

Existence and construction (Riesz representation). Fix $d \in \mathcal{D}$ and define a linear functional on $\mathcal{M}$ by

$$\ell_d(m) := (G(m), d)_{\mathcal{D}}. \tag{4.9}$$

Boundedness of G implies $\ell_d \in \mathcal{M}'$. By the Riesz theorem (Theorem 2.5) there exists a unique element $u \in \mathcal{M}$ such that $\ell_d(m) = (m, u)_{\mathcal{M}}$ for all m. We define $G^*(d) := u$.

Derivation of the commuting relation. Let $\mathcal{R}_{\mathcal{M}} : \mathcal{M}' \to \mathcal{M}$ and $\mathcal{R}_{\mathcal{D}} : \mathcal{D}' \to \mathcal{D}$ denote the Riesz maps, with inverses $\mathcal{R}_{\mathcal{M}}^{-1} : \mathcal{M} \to \mathcal{M}'$ and $\mathcal{R}_{\mathcal{D}}^{-1} : \mathcal{D} \to \mathcal{D}'$. Using the identity (Section 2.4, Section 2.5)

$$\langle \ell, x \rangle = (x, \mathcal{R}\ell)_X \qquad (\ell \in X',\ x \in X), \tag{4.10}$$

we compute, for arbitrary $m \in \mathcal{M}$ and $d \in \mathcal{D}$,

$$\begin{aligned}
(G(m), d)_{\mathcal{D}} &= \langle \mathcal{R}_{\mathcal{D}}^{-1} d, G(m) \rangle && \text{by (4.10) in } X = \mathcal{D} && (4.11)\\
&= \langle G'(\mathcal{R}_{\mathcal{D}}^{-1} d), m \rangle && \text{by the definition of } G' \text{ (4.4)} && (4.12)\\
&= (m, \mathcal{R}_{\mathcal{M}}\, G'\, \mathcal{R}_{\mathcal{D}}^{-1}(d))_{\mathcal{M}} && \text{by (4.10) in } X = \mathcal{M}. && (4.13)
\end{aligned}$$

Comparing with (4.8) and using uniqueness in the Riesz representation, we obtain the key relation

$$G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}. \tag{4.14}$$

API correspondence. In `pygeoinf`, this is exactly the rule that turns the Banach dual into a Hilbert adjoint:

$$\texttt{G.adjoint} = \texttt{domain.from_dual} \circ \texttt{G.dual} \circ \texttt{codomain.to_dual}. \tag{4.15}$$

Common Pitfall

If you are used to thinking of the matrix transpose as the adjoint, beware: that is only true in Euclidean space with the standard inner product. For general Hilbert spaces, $G^* \neq G^\top$. The correct relationship is $G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}$.

4.5 Matrix representations

All of the objects in this chapter are defined between infinite-dimensional spaces. Nevertheless, numerical algorithms require finite arrays. A *matrix representation* is nothing more than the coordinate description of the restriction of G to a finite-dimensional trial space and a finite-dimensional test space.

The key point for `pygeoinf` is that there are (at least) two natural matrix *views* of the same operator, and they differ in what the `rmatvec` operation means:

1. **Component view** (`galerkin=False`): `matvec` maps *primal components* to *primal components*, while `rmatvec` represents the *Banach dual* operator G' and therefore acts on *dual components*.

2. **Galerkin view** (`galerkin=True`): `matvec` maps *primal components* to *dual components* via a Riesz map in the codomain, and `rmatvec` represents the *Hilbert adjoint* operator G^*.

Both views are valid; which one you want depends on whether your algorithm is formulated in terms of duality pairings (G') or inner products (G^*).

Coordinates are not geometry. The maps `to_components/from_components` are *coordinate maps*. They depend on a chosen basis and are not assumed to be isometries. In particular, do *not* assume the basis is orthonormal; the inner products live separately, encoded by the Gram ("mass") matrices and Riesz maps.

4.5.1 The standard (component) matrix

Let $\mathcal{M}$ and $\mathcal{D}$ be Hilbert spaces, and choose finite- dimensional subspaces $\mathcal{M}_N = \mathrm{span}\{\phi_1, \ldots, \phi_N\} \subset \mathcal{M}$ and $\mathcal{D}_M = \mathrm{span}\{\psi_1, \ldots, \psi_M\} \subset \mathcal{D}$. Write $\pi_{\mathcal{M}} : \mathbb{R}^N \to \mathcal{M}_N$ for synthesis ("from components") and $\Pi_{\mathcal{D}} : \mathcal{D}_M \to \mathbb{R}^M$ for analysis ("to components").

The *standard component matrix* of G (with respect to these coordinate maps) is

$$\mathbf{G} \;:=\; \Pi_{\mathcal{D}} \circ G \circ \pi_{\mathcal{M}} \;\in\; \mathbb{R}^{M\times N}, \tag{4.16}$$

so that for a coefficient vector $\mathbf{c} \in \mathbb{R}^N$ we have $\mathbf{G}\,\mathbf{c} = \Pi_{\mathcal{D}}\big(G(\pi_{\mathcal{M}}\mathbf{c})\big)$, the *primal components* of the predicted data.

In this component view, `pygeoinf` uses `G.matrix(galerkin=False)` to build a matrix(-free) operator with `matvec` implementing (4.16).

What `rmatvec` means when `galerkin=False`. The reverse multiplication does *not* represent the Hilbert adjoint. Instead, it represents the *Banach dual* G':

$$\texttt{rmatvec}:\ \mathbb{R}^M \;\longrightarrow\; \mathbb{R}^N \quad \text{corresponds to} \quad G' : \mathcal{D}' \to \mathcal{M}'. \tag{4.17}$$

Concretely, its input is interpreted as *dual* components of some $d' \in \mathcal{D}'$ and its output is the *dual* component vector of $G'(d') \in \mathcal{M}'$. This is the correct object when your algorithm is written in terms of dual pairings (for example, building linear functionals or working directly in $\mathcal{D}'$).

4.5.2 The Galerkin matrix

When $\mathcal{M}$ and $\mathcal{D}$ are Hilbert spaces, we can convert primal vectors to dual vectors using the Riesz maps. Define the Gram (mass) matrix of the test space basis by

$$\mathbf{M}_{\mathcal{D}} \in \mathbb{R}^{M\times M}, \qquad (\mathbf{M}_{\mathcal{D}})_{ij} := (\psi_j, \psi_i)_{\mathcal{D}}. \tag{4.18}$$

This matrix is precisely the coordinate representation of the map $\mathcal{R}_{\mathcal{D}}^{-1} : \mathcal{D}_M \to \mathcal{D}'_M$: for $d \in \mathcal{D}_M$ with components $\mathbf{b} = \Pi_{\mathcal{D}}(d)$, the dual element $\mathcal{R}_{\mathcal{D}}^{-1} d \in \mathcal{D}'$ has dual components $\mathbf{M}_{\mathcal{D}}\,\mathbf{b}$.

The *Galerkin matrix* is then the coordinate representation of $\mathcal{R}_{\mathcal{D}}^{-1} \circ G$:

$$\tilde{\mathbf{G}} \;:=\; \Pi_{\mathcal{D}'} \circ \mathcal{R}_{\mathcal{D}}^{-1} \circ G \circ \pi_{\mathcal{M}} \;\in\; \mathbb{R}^{M\times N}. \tag{4.19}$$

In the common situation where $\Pi_{\mathcal{D}'}$ is the dual coordinate map associated with the basis $\{\psi_j\}$, we have the concrete relation

$$\tilde{\mathbf{G}} = \mathbf{M}_{\mathcal{D}}\,\mathbf{G}. \tag{4.20}$$

So `galerkin=True` does *not* merely change a convention; it changes which space the output coordinates live in (primal vs dual).

What `rmatvec` means when `galerkin=True`. In this view the reverse multiplication corresponds to the *Hilbert adjoint* G^*. More precisely, with `G.matrix(galerkin=True)`, the forward operation is $\mathbf{c} \mapsto \Pi_{\mathcal{D}'}(\mathcal{R}_{\mathcal{D}}^{-1} G(\pi_{\mathcal{M}}\mathbf{c}))$ and the reverse operation is

$$\mathbf{b} \mapsto \Pi_{\mathcal{M}'}(\mathcal{R}_{\mathcal{M}}^{-1} G^*(\pi_{\mathcal{D}}\mathbf{b})). \tag{4.21}$$

This is the representation that makes the adjoint appear as a transpose at the matrix level and is therefore the right choice for algorithms that depend on symmetry/self-adjointness.

Common Pitfall

Do not mix up the two matrix modes. With `galerkin=False`, `rmatvec` acts on *dual* components and represents G'. With `galerkin=True`, `rmatvec` acts on *primal* components and represents G^*. Treating one as the other leads to silent scaling errors (missing Gram/Riesz factors) and can break algorithms.

Where symmetry shows up. If $\mathcal{M} = \mathcal{D}$, the discretisation uses the same trial/test space, and G is self-adjoint ($G = G^*$), then the Galerkin matrix $\tilde{\mathbf{G}}$ is *symmetric* ($\tilde{\mathbf{G}} = \tilde{\mathbf{G}}^\top$). The standard component matrix $\mathbf{G}$ is generally *not* symmetric unless the chosen basis is orthonormal.

4.5.3 The discrete adjoint formula

Sometimes one wants a matrix that maps *primal* components in the data space to *primal* components in the model space, i.e.

$$G^*_{\text{comp}} \;:=\; \Pi_{\mathcal{M}} \circ G^* \circ \pi_{\mathcal{D}} \;\in\; \mathbb{R}^{N\times M}, \tag{4.22}$$

so that $\Pi_{\mathcal{M}}(G^*(d)) = G^*_{\text{comp}}\,\Pi_{\mathcal{D}}(d)$. In non-orthonormal coordinates this is *not* simply $\mathbf{G}^\top$.

Let $\mathbf{M}_{\mathcal{M}} \in \mathbb{R}^{N\times N}$ and $\mathbf{M}_{\mathcal{D}} \in \mathbb{R}^{M\times M}$ be the Gram matrices of the chosen bases, i.e. $(\mathbf{M}_{\mathcal{M}})_{ij} = (\phi_j, \phi_i)_{\mathcal{M}}$ and $(\mathbf{M}_{\mathcal{D}})_{ij} = (\psi_j, \psi_i)_{\mathcal{D}}$. Then the coordinate form of the commuting relation (4.14) yields the standard discrete adjoint identity

$$G^*_{\text{comp}} \;=\; \mathbf{M}_{\mathcal{M}}^{-1}\,\mathbf{G}^\top\,\mathbf{M}_{\mathcal{D}}. \tag{4.23}$$

This formula makes explicit why the transpose is only the adjoint in Euclidean coordinates: if $\mathbf{M}_{\mathcal{M}} = I$ and $\mathbf{M}_{\mathcal{D}} = I$ (orthonormal bases with the standard inner product), then $G^*_{\text{comp}} = \mathbf{G}^\top$. In general, the Gram matrices are unavoidable.

Connection to `LinearOperator.matrix`. The component view (`galerkin=False`) returns $\mathbf{G}$ and its `rmatvec` corresponds to G' (dual-to-dual), not to G^*_{comp}. The Galerkin view (`galerkin=True`) returns a matrix whose transpose implements the adjoint action at the level of coefficients (via (4.21)), and is the view in which self-adjointness manifests as matrix symmetry.

4.6 Worked example: `from_matrix`, dual vs adjoint, and Galerkin matrices

This chapter is written with an infinite-dimensional viewpoint: G is an operator between Hilbert spaces. The following example is therefore to be read as a *coordinate representation* used for computation (think: restricting to a finite trial/test space and choosing bases), not as a change in the underlying mathematical objects.

Set-up. Let the model space be Euclidean, $\mathcal{M} = \mathbb{R}^2$ with the standard inner product. Let the data space be a *mass-weighted* Euclidean space built from an underlying $H = \mathbb{R}^2$:

$$(u, v)_{\mathcal{D}} = u^\top W v, \qquad W = \mathrm{diag}(2, 3). \tag{4.24}$$

The geometry is encoded by the mass/Gram operator W (equivalently the Riesz maps). In particular, identifying a primal $d \in \mathcal{D}$ with a dual element $d' \in \mathcal{D}'$ introduces W.

Now define a tiny matrix-backed operator $G : \mathcal{M} \to \mathcal{D}$ via $A \in \mathbb{R}^{2\times 2}$:

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}. \tag{4.25}$$

```
import numpy as np

from pygeoinf.hilbert_space import EuclideanSpace,
    MassWeightedHilbertSpace
from pygeoinf.linear_operators import LinearOperator

V = EuclideanSpace(2)                        # model space (primal vectors
    are length-2 arrays)
H = EuclideanSpace(2)

W = LinearOperator.self_adjoint_from_matrix(H, np.diag([2.0, 3.0]))
W_inv = LinearOperator.self_adjoint_from_matrix(H, np.diag([0.5,
    1.0/3.0]))
D = MassWeightedHilbertSpace(H, W, W_inv)  # data space with <u,v>_D = u^
    T W v

A = np.array([[1.0, 2.0],
              [0.0, 1.0]])
G = LinearOperator.from_matrix(V, D, A, galerkin=False)

m = np.array([1.0, -1.0])
d = G(m)                  # primal vector in D

dp = D.to_dual(d)          # dual element in D'  (a LinearForm)
mp = G.dual(dp)            # dual element in V'
g  = G.adjoint(d)          # primal vector in V

# Consistency check (Riesz-first identity):
# g == V.from_dual( G.dual( D.to_dual(d) ) )
```

Type distinctions (what each map consumes). The same operator gives rise to three different (typed) actions:

- G acts on *primal vectors* $m \in \mathcal{M}$ and returns a primal vector $d \in \mathcal{D}$.
- $G' \equiv$ `G.dual` acts on *linear forms* $d' \in \mathcal{D}'$ and returns a linear form $m' \in \mathcal{M}'$.
- $G^* \equiv$ `G.adjoint` acts on *primal vectors* $d \in \mathcal{D}$ and returns a primal vector in $\mathcal{M}$.

Even in finite dimensions these are not interchangeable: the identifications between primal and dual spaces are part of the Hilbert-space geometry.

One place where dual and adjoint differ. In coordinates, the Banach-dual action is

$$\text{if } d' \in \mathcal{D}' \text{ has dual components } \mathbf{c}_{d'}, \qquad \Pi_{\mathcal{M}'}(G'(d')) = A^\top \mathbf{c}_{d'}. \tag{4.26}$$

But the Hilbert adjoint involves the Riesz identification in the data space. With $(u, v)_{\mathcal{D}} = u^\top W v$ we have, conceptually, $\mathcal{R}_{\mathcal{D}}^{-1}(d) \leftrightarrow W\, d$. Thus the adjoint acts as

$$\Pi_{\mathcal{M}}(G^*(d)) = A^\top W\, \Pi_{\mathcal{D}}(d), \tag{4.27}$$

which differs from (4.26) by the W factor. This is exactly why the library keeps `dual` and `adjoint` separate: G' is a dual-to-dual map, while G^* is a primal-to-primal map.

The two matrix modes in this example. By default, `G.matrix(...)` returns a matrix-free SciPy `LinearOperator` (`dense=False`). To materialise a dense array for inspection, use `dense=True`. For the same operator, the two dense matrix views are

$$\texttt{G.matrix(dense=True, galerkin=False)} = A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \tag{4.28}$$

$$\texttt{G.matrix(dense=True, galerkin=True)} = W A = \begin{bmatrix} 2 & 4 \\ 0 & 3 \end{bmatrix}. \tag{4.29}$$

The component view returns the component-to-component map, while the Galerkin view left-multiplies by the codomain mass/Gram matrix (equivalently, post-composes with $\mathcal{R}_{\mathcal{D}}^{-1}$) so that the reverse multiply corresponds to the Hilbert adjoint. This is the same content as the abstract identity $G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}$, written in coordinates.

4.7 Operators as stacks of linear forms

pygeoinf API

Given linear forms $\ell_i \in \mathcal{M}'$, construct the forward operator via `LinearOperator.from_linear_forms([`$\ell_1, \ldots, \ell_M$

4.8 Constructing operators in pygeoinf

The class `LinearOperator` is designed to keep the type distinctions visible: the forward map acts on primal vectors, the Banach-dual map acts on linear forms, and the Hilbert adjoint acts again on primal vectors. In code these are exposed as `G`, `G.dual`, and `G.adjoint`.

The primary constructor. The most flexible construction route is `LinearOperator(domain, codomain, mapping, ...)`. The callables you supply are actions on *elements* of the relevant spaces, not on component vectors.

pygeoinf API

```
LinearOperator( domain, codomain, mapping, dual_mapping=..., adjoint_mapping=...,
dual_base=..., adjoint_base=... )
```

What the arguments mean. Let $G : \mathcal{M} \to \mathcal{D}$ be the intended forward operator. Then the constructor arguments have the following semantics.

- `mapping`: implements the primal action $G(x) \in \mathcal{D}$ for $x \in \mathcal{M}$. This is what runs when you call `G(x)`.
- `dual_mapping`: implements the dual action $G'(\ell) \in \mathcal{M}'$ for $\ell \in \mathcal{D}'$. In the library, a dual element is represented by a `LinearForm` and is created via `space.to_dual(...)`. This is what runs when you call `G.dual(yp)`.
- `adjoint_mapping`: implements the Hilbert adjoint action $G^*(y) \in \mathcal{M}$ for $y \in \mathcal{D}$ (primal vector in the codomain). This is what runs when you call `G.adjoint(y)`.
- `dual_base`, `adjoint_base`: internal book-keeping used when constructing `G.dual` and `G.adjoint`. Most users should never set these explicitly.

What gets derived if you only provide one mapping. The library prefers you to supply at least one of `dual_mapping` or `adjoint_mapping`. If you provide only one, the other is derived using the Riesz maps (i.e. `to_dual` and `from_dual`) of the spaces, exactly mirroring the identity $G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}$.

pygeoinf API

If you provide `dual_mapping` (i.e. G'), the adjoint is derived as `G.adjoint(y) = domain.from_dual( G.dual( codomain.to_dual(y) ) )`.

If you provide `adjoint_mapping` (i.e. G^*), the dual is derived as `G.dual(yp) = domain.to_dual( G.adjoint( codomain.from_dual(yp) ) )`.

If you provide neither, `G.dual` falls back to the definition $G'(\ell) = \ell \circ G$ (composition of linear forms with the forward map) and `G.adjoint` is then obtained via the Riesz maps. This is mathematically correct, but can be much slower than a purpose-built adjoint/dual action.

When `mapping` alone is enough. Supplying only `mapping` is fine when you will *only* evaluate the forward operator and never need `G.dual` or `G.adjoint` (e.g. a quick diagnostic or a forward simulation). It is also acceptable for very small problems where the default derived actions are not performance limiting.

When you should also provide `dual_mapping` or `adjoint_mapping`. Most inverse-problem workloads repeatedly apply an adjoint (or dual): gradients, Gauss–Newton steps, normal equations, Krylov solvers, and many preconditioners all rely on `G.adjoint` or `G.dual`. In those cases, you should implement at least one of these actions explicitly.

- Prefer supplying `adjoint_mapping` when you can write an algorithmic reverse operator that maps a data-space vector to a model-space vector. This is usually the most direct route for optimisation.
- Prefer supplying `dual_mapping` when your operator is naturally described as a collection of linear functionals (or you already work in dual space), so that applying G' is simpler than applying G^*.

Common Pitfall

Do not assume "transpose equals adjoint" unless your geometry is Euclidean. In `pygeoinf`, `G.dual` (Banach dual) and `G.adjoint` (Hilbert adjoint) are different objects in general, related by the Riesz maps `to_dual`/`from_dual`.

Formal adjoint vs Hilbert adjoint (and why weights matter). In many PDE and FEM derivations one first writes a *formal adjoint* $G^\dagger$ with respect to a simple, unweighted L^2 pairing. However, the operator *actually used* by algorithms in the library is the *Hilbert adjoint* G^* determined by the inner products on $\mathcal{M}$ and $\mathcal{D}$. When either space is a `MassWeightedHilbertSpace`, its inner product is defined via an underlying space X and a mass operator M: $(u, v)_Y = (Mu, v)_X$. The Riesz map is modified accordingly (see `MassWeightedHilbertSpace.to_dual` and `.from_dual`).

Concretely, if $\mathcal{M}$ and $\mathcal{D}$ are mass-weighted by operators $M_\mathcal{M}$ and $M_\mathcal{D}$ (identity in the unweighted case), then the Hilbert adjoint relates to the formal adjoint by the mass-weighted conjugation

$$G^* \;=\; M_\mathcal{M}^{-1} \circ G^\dagger \circ M_\mathcal{D}. \tag{4.30}$$

This is exactly what `LinearOperator.from_formal_adjoint` encodes: you implement an operator (and its adjoint) on the underlying unweighted spaces, then *lift* it to the correct weighted geometry so that `G.adjoint` matches the Hilbert adjoint dictated by the mass operators.

pygeoinf API

If you already have an operator on the underlying spaces ("formal" geometry), lift it to mass-weighted spaces via `LinearOperator.from_formal_adjoint(domain, codomain, op_L2)`.

Safe factory routes (recommended when applicable). Whenever you have a clean, explicit representation, prefer the dedicated factories. They avoid ambiguity about whether a matrix is interpreted in standard or Galerkin form and they return specialized subclasses for better performance.

pygeoinf API

`LinearOperator.from_matrix(domain, codomain, A, galerkin=False)` wraps a standard component matrix A.

`LinearOperator.from_matrix(domain, codomain, K, galerkin=True)` wraps a Galerkin matrix K (already mass-weighted in the codomain).

`LinearOperator.self_adjoint_from_matrix(domain, K)` constructs a self-adjoint operator, interpreting `K` as Galerkin.

`LinearOperator.from_linear_forms([`$\ell_1, \ldots, \ell_N$`])` builds an operator whose rows are linear forms $\ell_i \in \mathcal{M}'$.

4.9 Advanced routines: randomized spectral surrogates

Beyond construction and basic algebra, `LinearOperator` provides a small set of *advanced routines* that build cheap, low-rank spectral surrogates of an operator. These are primarily intended for

diagnostics ("what is the effective rank/spectrum here?"), for quick *surrogate models* in experiments, and for developing *preconditioning intuition* before you commit to a full-scale solver.

`random_svd`. `G.random_svd(size_estimate, ...)` returns three operator factors (U, Σ, V) with an internal Euclidean space $\mathbb{R}^r$ (where r is the achieved rank returned by the routine) such that the composition $G \approx U\,\Sigma\,V$ acts as a rank-r approximation of G in the chosen matrix view. Concretely, $V : \mathcal{M} \to \mathbb{R}^r$ maps a model to low-dimensional coordinates, Σ is a diagonal operator of approximate singular values, and $U : \mathbb{R}^r \to \mathcal{D}$ maps back into the data space. Use this when you want a quick picture of dominant input/output directions, cheaply compress an operator for prototyping, or build a low-rank surrogate that is fast to apply repeatedly.

`random_eig`. `A.random_eig(size_estimate, ...)` returns an approximate eigensystem for a square operator (an automorphism). It produces an "expansion" operator $U : \mathbb{R}^r \to \mathcal{M}$ together with a diagonal operator of eigenvalues Λ so that, in the intended self-adjoint setting, $A \approx U\,\Lambda\,U^*$. This is useful for sanity-checking spectral behaviour (e.g. identifying dominant modes, verifying approximate symmetry/self-adjointness in practice), and for building simple spectral preconditioner sketches.

`random_cholesky`. `C.random_cholesky(size_estimate, ...)` returns an operator factor $F : \mathbb{R}^r \to \mathcal{M}$ intended for positive-definite, self-adjoint operators such that $C \approx F\,F^*$. It is a convenient way to obtain an approximate "square root" of a covariance or Gram-type operator for exploratory uncertainty quantification, cheap sampling surrogates, and preconditioning intuition.

Common Pitfall

These routines are *randomized* and *approximate*. Results depend on the random seed and on whether the operator is well captured by a low-rank surrogate.

More subtly, results also depend on *geometry choices*: in `pygeoinf` the same operator has different coordinate-level matrix views (component vs Galerkin; Section 4.5). A "spectrum" computed in a component view is not, in general, the Hilbert-space spectrum associated with your inner product. Finally, operator properties matter: `random_eig` is intended for self-adjoint/symmetric behaviour in the relevant inner product, and `random_cholesky` assumes positive-definiteness; if these conditions are violated, the outputs may be misleading even if the code runs.

4.10 The full commutative diagram

The objects and maps introduced in this chapter — the forward operator G, its Banach dual G', its Hilbert adjoint G^*, the analysis/synthesis (coordinate projection/inclusion) maps Π_N, and the primal and dual Riesz maps $\mathcal{R}$ — fit together into a single commutative diagram. Figure 4.1 shows the full picture: the outer square lives in the infinite-dimensional spaces $\mathcal{M}$ and $\mathcal{D}$; the inner rows involve the finite-dimensional surrogates $\mathbb{R}^N$ and $\mathbb{R}^{N_d}$ used in computations.

The central identity threading the diagram is

$$G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}, \tag{4.31}$$

which says that the Hilbert adjoint is the Banach dual conjugated by the Riesz maps of the two spaces. The blue arrows highlight the maps that depend on the inner-product geometry of $\mathcal{M}$ and

$\mathcal{D}$: any change in those inner products (e.g. using a mass-weighted space) affects $\mathcal{R}_{\mathcal{M}}$, $\mathcal{R}_{\mathcal{D}}$, and therefore G^*, while leaving G' and the coordinate maps untouched.

$$
\begin{tikzcd}
\mathcal{M} \arrow[rrr, "G"] \arrow[dr, "\Pi_N"] \arrow[ddd, bend right, "\mathcal{R}_{\mathcal{M}}^{-1}"'] & & & \mathcal{D} \arrow[lll, bend right, "G^*"'] \arrow[dl, "\Pi_{\mathcal{D}}"] \arrow[ddd, bend left, "\mathcal{R}_{\mathcal{D}}^{-1}"] \\
& \mathcal{M}_N \arrow[ul, "i_{\mathcal{M}}"] \arrow[d, shift left, "\mathcal{R}_{\mathcal{M}_N}"] & \mathbb{R}^N \arrow[l, "\pi_N"'] \arrow[r, "\mathbf{G}"] \arrow[d, shift left, "\mathcal{R}_{\mathbb{R}^N}^{-1}"] & \mathbb{R}^{N_d} \arrow[l, bend right, "\mathbf{G}^*"'] \arrow[ur, bend left, "\pi_{\mathcal{D}}"] \arrow[d, shift left, "\mathcal{R}_{\mathbb{R}^{N_d}}^{-1}"] \\
& \mathcal{M}'_N \arrow[u, shift left, "\mathcal{R}_{\mathcal{M}_N}^{-1}"] \arrow[r, "\pi'_N"] & (\mathbb{R}^N)' \arrow[l, bend right, "(\pi'_N)^*"'] \arrow[u, shift left, "\mathcal{R}_{\mathbb{R}^N}"] \arrow[dl, "\Pi'_N"] & (\mathbb{R}^{N_d})' \arrow[l, "\mathbf{G}'"] \arrow[u, shift left, "\mathcal{R}_{\mathbb{R}^{N_d}}"] \arrow[dl, bend left, "\Pi'_{\mathcal{D}}"] \\
\mathcal{M}' \arrow[ur, "i'_{\mathcal{M}}"] \arrow[rrr, "G'"'] & & & \mathcal{D}' \arrow[ul, "\pi'_{\mathcal{D}}"']
\end{tikzcd}
$$

Figure 4.1: Summary of all maps between $\mathcal{M}$, $\mathcal{D}$, their duals, and their finite-N surrogates $\mathbb{R}^N$, $\mathbb{R}^{N_d}$. Blue arrows are geometry-dependent (they involve the Riesz maps of $\mathcal{M}$ or $\mathcal{D}$ and hence change when the inner products change). The key commuting identity is $G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}$ (outer square, going top-right $\to$ bottom $\to$ top-left vs top $\to$ top-right). The inner rows give the finite-dimensional computational surrogates and how they interlock with the infinite-dimensional picture.

Chapter 5

Nonlinear Operators and Forms

5.1 NonLinearOperator

5.2 NonLinearForm

5.3 Gradients and Hessians

Part II

Geometry and Structure

Chapter 6

Subsets and Convex Sets

6.1 Why convex sets?

Why this matters

Every piece of prior information you have about your model — bounds on parameters, energy constraints, smoothness requirements — defines a set of admissible models. Convexity is the mathematical property that makes these constraints computationally tractable.

6.2 The Subset interface

6.3 Balls and ellipsoids

6.4 Half-spaces and polyhedral sets

6.5 Degenerate sets: EmptySet and FullSpace

6.6 Set operations

Chapter 7

Subspaces

7.1 Motivation: equality constraints

7.2 Orthogonal projectors

pygeoinf API

`OrthogonalProjector.from_basis(space, [v1, v2, ...])` constructs the projection $P = \sum_i \hat{v}_i \hat{v}_i^\top$ from spanning vectors.

7.3 Linear subspaces

7.4 Affine subspaces

7.4.1 Construction from tangent bases

7.4.2 Construction from linear equations

7.5 Tangent spaces and dimension

Chapter 8

Convex Analysis and Support Functions

Why this matters

Convex analysis provides a complete, probability-free framework for answering the question: given my data and prior constraints, what can I confidently say about the model? The support function is the central object that makes this possible.

8.2 Support functions: intuition

8.3 Formal definition and properties

8.4 Subgradients and support points

8.5 Concrete support functions

8.5.1 BallSupportFunction

8.5.2 EllipsoidSupportFunction

8.5.3 PolyhedralSupportFunction

8.5.4 SobolevBallSupportFunction

8.6 Combinators: building complex sets from simple ones

8.6.1 Infimal convolution (Minkowski sum)

8.6.2 Support function sum (intersection)

8.6.3 Scaled sums

8.7 The dual master equation

Chapter 9

Gaussian Measures

9.1 Why Gaussian measures?

Why this matters

If your prior knowledge about the model is naturally described by a mean and covariance, and your forward operator is linear, and your measurement noise is Gaussian, then the full posterior distribution is Gaussian and can be computed in closed form. This is what `GaussianMeasure` provides.

9.2 Gaussian measures on Hilbert spaces

9.3 Sampling

9.4 Push-forward and marginals

9.5 Conditioning (Bayesian update)

Part III

Problems and Algorithms

Chapter 10

Forward Problems

10.1 The forward modelling framework

10.2 ForwardProblem and LinearForwardProblem

pygeoinf API

`LinearForwardProblem(model_space, data_space, forward_operator, data_error_measure)` bundles the complete forward problem specification.

Chapter 11

Deterministic Inversion

11.1 Least-squares inversion

11.2 Minimum-norm inversion

Why this matters

The minimum-norm solution depends on the inner product: change the norm, change the answer. This is why the careful treatment of Hilbert spaces in Chapter 2 pays off here.

11.3 Constrained inversions

11.4 Connection to the dual master equation

Chapter 12

Bayesian Inversion

12.1 The Bayesian framework for inverse problems

12.2 Linear Bayesian inversion

pygeoinf API

`LinearBayesianInversion(forward_problem, prior)` computes the posterior Gaussian measure. Access via `.invert(data)`.

12.3 Constrained Bayesian inversion

12.4 Posterior uncertainty

Chapter 13

Backus–Gilbert and DLI Inference

13.1 The inference problem

13.2 Backus–Gilbert estimators

13.3 The admissible property set $\mathcal{U}$

13.4 Computing $\mathcal{U}$ via the dual master equation

13.5 Polytope outer approximation

Chapter 14

Convex Optimisation

14.1 Overview of optimisation in pygeoinf

14.2 Linear solvers

14.3 Convex optimisation algorithms

14.4 Preconditioners

Part IV

Computational Methods and Visualisation

Chapter 15

Computational Utilities

15.1 Randomised matrix decompositions

15.2 Direct sum operators

15.3 Parallel computation

15.4 Auxiliary functions

Chapter 16

Visualisation

16.1 The SubspaceSlicePlotter

16.2 1D slices

16.3 2D slices

16.4 3D slices

16.5 Worked example: visualising the admissible property set

Appendices

Appendix A

Convex Analysis Primer

A.1 Convex sets and functions

A.2 Separation theorems

A.3 Conjugate (Fenchel–Legendre) duality

A.4 Subdifferentials

A.5 Infimal convolution

Appendix B

Functional Analysis Prerequisites

B.1 Banach spaces

B.2 Weak and weak-* topologies

B.3 Compact operators

B.4 Spectral theory for self-adjoint operators

Appendix C

Deferred Proofs

Appendix D

Complete API Quick Reference

D.1 hilbert_space module

D.2 linear_forms module

D.3 linear_operators module

D.4 subsets module

D.5 subspaces module

D.6 convex_analysis module

D.7 gaussian_measure module

D.8 forward_problem module

D.9 inversion module

D.10 linear_bayesian module

D.11 linear_optimisation module

D.12 backus_gilbert module

D.13 visualisation module

Bibliography

[1] David Al-Attar and Ophelia Crawford. Linear inference problems with deterministic constraints. *Geophysical Journal International*, 2021. doi: 10.1093/gji/ggab414.

[2] George Backus. Inference from inadequate and inaccurate data, I. *Proceedings of the National Academy of Sciences*, 65(1):1–7, 1970. doi: 10.1073/pnas.65.1.1.

[3] George Backus. Inference from inadequate and inaccurate data, III. *Proceedings of the National Academy of Sciences*, 67(1):282–289, 1970. doi: 10.1073/pnas.67.1.282.

[4] George Backus. Inference from inadequate and inaccurate data, II. *Proceedings of the National Academy of Sciences*, 65(2):281–287, 1970. doi: 10.1073/pnas.65.2.281.

[5] Andrew M. Stuart. Inverse problems: A Bayesian perspective. *Acta Numerica*, 19:451–559, 2010. doi: 10.1017/S0962492910000061.